

AI 기반 시계열 데이터 분석 및 AI 모델 운영

서울시립대학교 전자전기컴퓨터공학부 엄대호 교수

INSTRUCTOR

강사 소개

임대호

서울시립대학교
전자전기컴퓨터공학부

조교수

수상

강의우수교수상 (2025)

서울시립대학교

학력

공학박사 · 전기·정보공학

서울대학교 (서울, 대한민국)

2019.03 – 2024.08

공학사 · 전기전자전파공학

고려대학교 (서울, 대한민국)

2013.03 – 2019.02

경력

조교수

2025.09 – 현재

서울시립대학교 전자전기컴퓨터공학부

스태프 엔지니어

2025.01 – 2025.08

삼성전자 AI센터 공정/설비 AI Lab

스태프 엔지니어

2024.09 – 2024.12

SAIT(삼성종합기술원) Machine Learning Technical Unit

주요 강의

SK그룹 ML Engineer 과정 · 현대자동차 AI Boosting Camp 과정 · 삼성전자 AI전문가 과정

언론 보도

[중앙일보] 서울시립대 임대호 교수 연구팀, 세계 최고 권위의 머신러닝 학회 'ICML 2026' 논문 채택

[한겨레] 서울시립대 임대호 교수 연구팀, 인공지능 분야 세계 최고 권위 'CVPR 2026' 논문 채택

생성형 AI와 바이브 코딩

코딩 지식 0에서 시작해 '내 업무용 도구'를 직접 만들어 봅니다

PART 1 · Why

- 생성형 AI와 바이브 코딩의 개념을 이해한다
- AI로 생산성이 어떻게 바뀌는지 사례로 안다
- 내 업무 중 어디에 쓸 수 있을지 판단한다

PART 2 · How

- AI에게 원하는 것을 정확히 전달한다
- 말로 만들고 고치는 과정을 직접 본다
- 간단한 도구를 하나 완성해 본다

오늘의 원칙 — "완벽한 시스템"이 아니라 "당장 쓸 수 있는 도구"

80% 완성도의 도구를 오늘 만들어 내일부터 쓰는 것이, 100% 완성도의 시스템을 6개월 뒤에 받는 것보다 낫습니다. 이 과정은 시스템 개발이 아니라 '만드는 방법'을 익히는 교육입니다.

잠깐, 지난주를 떠올려 볼까요?

"지난 한 주 동안, 똑같은 손동작을 반복한 업무가 있었나요?"

복사 · 붙여넣기

여기저기 흩어진 값을 하나의 엑셀로 옮겨 담기

취합 · 대조

여러 사람에게 받은 양식을 합치고 오탈자 확인하기

같은 질문 응대

"그거 어떻게 됐어요?" "그 자료 어디 있어요?"

이런 업무는 '사람이 잘해서' 끝나는 일이 아니라, '도구가 없어서' 사람이 하고 있는 일입니다.

오늘은 그 도구를, 개발팀에 요청하지 않고 직접 만들어 봅니다.

PROBLEM

업무에서 시간이 새는 3곳

전문성이 필요한 일이 아니라, 손이 많이 가는 일이 시간을 잡아먹습니다

① 입력 · 이관 (Input)

메일·메신저·문서로 받은 내용을 사람이 다시 시스템이나 엑셀에 옮겨 적는 구간

② 취합 · 정리 (Aggregate)

여러 곳에 흩어진 파일을 모아 형식을 맞추고 합계를 내는 구간

③ 조회 · 응대 (Lookup)

이미 존재하는 정보를 담당자가 찾아 알려 주느라 소모되는 구간

→ 세 구간 모두 '규칙이 정해져 있고, 반복되며, 판단이 거의 필요 없는' 일입니다. 자동화 1순위입니다.

생성형 AI, 한 장으로 이해하기

STEP 1

엄청난 양의 글을 학습

책·문서·코드 등 방대한 텍스트에서 언어의
패턴을 익힘

STEP 2

다음에 올 말을 예측

앞의 내용을 보고 가장 그럴듯한 다음 단어
를 계속 이어 붙임

STEP 3

결과물을 '생성'

글·표·요약뿐 아니라 코드도 같은 원리로 만
들어 냄

잘하는 일

- 정해진 형식으로 정리·변환·요약하기
- 예시를 주면 비슷하게 만들어 내기
- 코드처럼 규칙이 명확한 언어 다루기

주의할 일

- 모르는 것도 그럴듯하게 지어냄 (환각)
- 최신 정보·내부 규정은 알려 줘야 함
- 결과는 반드시 사람이 확인해야 함

CONCEPT 2

AI를 쓰는 방식은 3단계로 발전했습니다

대부분의 직장인은 아직 1단계에 머물러 있습니다

LV.1 물어보고 받아쓰기

"보고서 초안 써 줘" — 결과를 복사해 내가 다시 붙여넣기. 매번 반복해야 함.

LV.2 내 문서·데이터 다루기

파일을 주고 요약·분류·표 정리를 맡김. 여전히 한 번 쓰고 끝나는 작업.

LV.3 도구 자체를 만들게 하기

"이 일을 매번 해 주는 프로그램을 만들어 줘" — 한 번 만들면 계속 재사용. ← 바이트 코딩

핵심 차이 — 1·2단계는 '결과물'을 받고, 3단계는 '결과물을 만들어 주는 기계'를 받습니다.

바이브 코딩(Vibe Coding)이란?

"코드를 직접 쓰는 대신, 하고 싶은 일을 말로 설명하면
AI가 코드를 만들고 고쳐 주는 방식의 개발"

2025년 초 AI 연구자 안드레이 카파시(Andrej Karpathy)가 처음 쓴 표현으로, 지금은 업계 공용어가 되었습니다.

1

말로 지시한다

프로그래밍 문법이 아니라 우리말 문장으로
요구사항을 전달합니다.

2

결과로 확인한다

코드를 읽고 이해하기보다, 실제로 돌려
보고 맞는지 판단합니다.

3

대화로 고친다

"버튼을 오른쪽으로", "합계도 보여 줘"
— 계속 말로 수정합니다.

주의 — '바이브'는 감으로 만든다는 뜻이지, 검증하지 않아도 된다는 뜻이 아닙니다. 결과 확인은 사람의 몫입니다.

COMPARE

지금까지의 방식과 무엇이 다른가

구분	기존 방식	바이브 코딩
시작하는 법	요구사항 정의서 작성 → 개발 요청	AI에게 하고 싶은 일을 문장으로 설명
필요한 지식	프로그래밍 언어, 개발 환경	업무 지식 + 설명하는 능력
첫 결과까지	수 주 ~ 수 개월	수 분 ~ 수 시간
수정 방법	변경 요청 → 일정 협의 → 재작업	그 자리에서 말로 수정
적합한 대상	전사 시스템, 대규모·고신뢰 서비스	팀·개인용 소규모 도구, 시제품

둘은 경쟁 관계가 아닙니다. 큰 시스템은 여전히 개발 조직이, 작은 도구는 실무자가 직접 만드는 '역할 분담'이 생긴 것입니다.

WHY NOW

왜 '비전공자'에게 더 큰 기회인가

가장 어려운 부분이 이미 해결되었기 때문입니다

기존

무엇이 어려웠나

- 업무를 아는 사람은 만들 줄 몰랐고
- 만들 줄 아는 사람은 업무를 몰랐다
- 그 간극을 메우려 요구사항 정의서와 회의가 반복됨
- 결국 원했던 것과 다른 결과물이 나옴

지금

무엇이 달라졌나

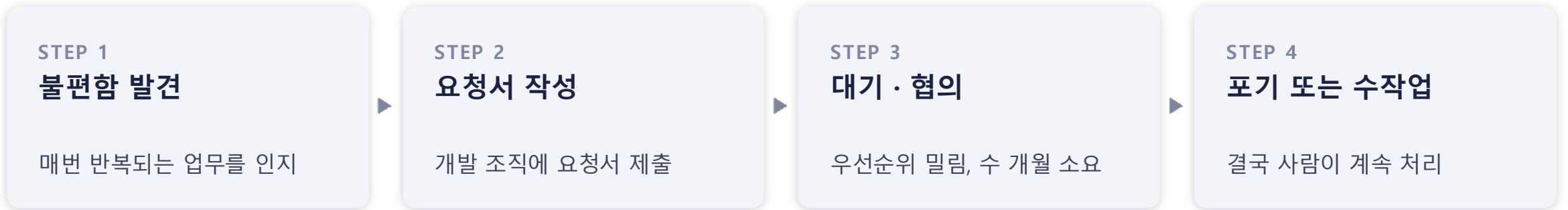
- 코드를 쓰는 능력은 AI가 대신함
- 남은 것은 '무엇을 만들지 아는 능력'
- 그건 실무자가 가장 잘 아는 영역
- 전달의 간극 없이 바로 만들어 확인 가능

이제 경쟁력은 '코딩 실력'이 아니라 '업무를 정확히 설명하는 능력'입니다.

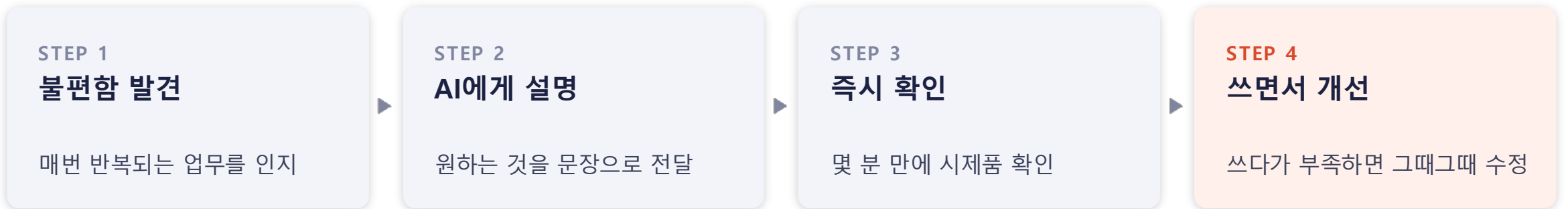
CHANGE

일하는 방식이 이렇게 바뀝니다

AS-IS · 지금까지



TO-BE · 앞으로



가장 큰 변화는 속도가 아니라 '포기하지 않게 되는 것'입니다.

CASE 1

생산성 향상 사례 ① 개인 업무

특별한 사람이 아니라, 일반 실무자들이 만들고 있는 것들

엑셀 취합 자동화

여러 개의 양식 파일을 한 폴더에 넣으면
하나의 통합 표로 합쳐 주는 도구

3시간 → 3분

반복 문서 생성기

이름·기간·금액만 입력하면 정해진 서식의
문서를 자동으로 만들어 주는 도구

건당 20분 → 1분

데이터 현황판

엑셀 파일을 올리면 그래프와 요약 지표를
한 화면에 보여 주는 도구

보고 준비 부담 감소

공통점 3가지

- 대단한 기술이 아니라 '내 손이 가장 많이 가는 일'을 골랐다
- 처음부터 완벽하게 만들지 않고, 하나의 기능부터 시작했다
- 만든 사람이 곧 사용자였기 때문에 요구사항 전달 손실이 없었다

CASE 2

생산성 향상 사례 ② 팀 단위 활용

업무 종류는 달라도, 만드는 도구의 형태는 비슷합니다

상황	만든 도구	기존 방식	효과
여러 명에게 자료를 받을 때	제출 현황 자동 집계판	메일 회신을 세어 수기 관리	확인·독촉 시간 감소
여러 건을 동시에 진행할 때	업무 진행 현황판	공용 엑셀 동시 편집 충돌	상태 혼선 해소
같은 서식을 반복 작성할 때	정형 문서 생성기	매번 이전 파일 복사·수정	건당 작성 시간 단축
응답·의견을 분류할 때	응답 분류·집계 화면	수백 건을 눈으로 읽고 분류	분류 시간 대폭 감소
마감이 여러 개일 때	마감 임박 항목 알림 목록	담당자 기억과 캘린더에 의존	누락 위험 제거

모두 '기존 시스템을 대체'한 것이 아니라, 시스템과 사람 사이의 빈틈을 메운 작은 도구들입니다.

숫자로 보는 변화

실제 도입 사례에서 공통적으로 나타나는 패턴

**3시간
→ 5분**

반복 자료 취합
(작업 자동화)

**수 개월
→ 반나절**

아이디어에서
시제품까지 걸리는 시간

0줄

실무자가 직접 작성한
코드의 양

재사용

한 번 만든 도구는
계속 사용

다만, 여기에는 조건이 있습니다

- ① 업무를 정확히 아는 사람이 직접 만들 것 ② 범위를 작게 시작할 것 ③ 결과를 사람이 반드시 검증할 것.
이 세 가지가 빠지면 '빨리 만든 쓸모없는 것'이 됩니다.

무엇까지 만들 수 있나

기대치를 정확히 알아야 실망하지 않고 오래 씁니다

가능

지금 바로 만들 수 있는 것

- 입력 화면 + 목록 + 검색이 있는 관리 도구
- 엑셀 파일을 읽어 집계·그래프로 보여 주는 화면
- 정해진 규칙으로 문서·메일 초안을 만드는 도구
- 체크리스트·설문·신청 양식 같은 간단한 화면
- 반복 작업을 대신하는 소규모 자동화

주의

아직 신중해야 하는 것

- 사내 기간제 시스템과의 직접 연동
- 민감 정보를 저장·유통하는 실 운영 시스템
- 여러 부서가 동시에 쓰는 전자 서비스
- 법적 효력·감사 대상이 되는 기록 관리
- 장애 시 업무가 멈추는 핵심 업무 시스템

오른쪽 영역은 '불가능'이 아니라 '개발·보안 조직과 함께 가야 하는 영역'입니다.

CAUTION

시작하기 전 알아야 할 3가지



정보 보안

회사 내부 자료나 민감한 개인정보를 그대로 입력하지 않습니다. 샘플 데이터로 만들고, 실제 데이터는 완성된 도구에서 만 다룹니다.



검증 없는 신뢰

AI는 그럴듯한 오답을 냅니다. 계산식과 집계 결과는 반드시 손으로 검산한 뒤 사용합니다.



유지보수 공백

만든 사람이 자리를 옮기면 아무도 못 고치는 도구가 됩니다. 만든 과정과 요구사항을 기록으로 남깁니다.

대응 원칙 — "가짜 데이터로 만들고, 진짜 데이터로 검증하고, 기록으로 남긴다"

오늘 시연과 실습도 전부 샘플 데이터로 진행합니다.

여기까지 정리하면

1

업무 시간은 '입력·취합·조화'에서 샌다

판단이 필요 없는 반복 구간이 자동화 1순위입니다.

2

AI 활용의 3단계는 '도구를 만들게 하기'에서 완성된다

결과물이 아니라 결과물을 만드는 도구를 얻습니다.

3

이제 필요한 능력은 코딩이 아니라 '설명하는 능력'이다

업무를 가장 잘 아는 실무자가 가장 유리한 위치에 있습니다.

4

단, 보안·검증·기록 3가지 원칙은 예외 없이 지킨다

샘플 데이터로 만들고, 결과는 손으로 검산합니다.

AI로 만드는 도구는 3가지 유형이 있습니다

무엇을 만들 것인지에 따라 적합한 도구가 다릅니다

① 대화형 (챗봇)

질문하면 코드를 알려 줌. 결과를 내가 복사해 옮겨야 함.

일반 AI 채팅 서비스

② 편집기 내장형

개발용 프로그램 안에서 코드 작성을 도와줌. 개발자에게 적합.

코드 에디터 확장 도구

③ 에이전트형

파일 생성·수정·실행까지 스스로 수행. 비전공자에게 가장 적합.

Claude Code

오늘 사용할 도구 — Claude Code (에이전트형)

비전공자에게 가장 큰 장벽은 '코드를 어디에 붙여넣고 어떻게 실행하는가'입니다. 에이전트형 도구는 그 과정을 대신 처리해 주기 때문에, 우리는 '무엇을 만들지'에만 집중할 수 있습니다.

Claude Code, 이렇게 이해하세요

"내 컴퓨터의 특정 폴더 안에서, 내 지시대로 파일을 만들고 고쳐 주는 AI 동료"

1. 폴더 단위로 일한다

지정한 작업 폴더 안의 파일만 읽고 씁니다. 범위가 명확해 안전합니다.

2. 직접 실행하고 확인한다

만든 것을 실행해 오류를 스스로 발견하고 고치는 과정을 반복합니다.

3. 대화가 이어진다

"아까 그 화면에 검색 기능 추가해 줘"처럼 맥락을 유지한 채 수정합니다.

4. 승인을 받고 움직인다

파일 수정·명령 실행 전 사용자에게 확인을 요청하도록 설정할 수 있습니다.

바이트 코딩은 이 4단계의 반복입니다

STEP 1

① 설명한다

무엇을 만들지 문장으로 전달
"~하는 화면을 만들어 줘"

STEP 2

② 생성된다

AI가 파일을 만들고
실행 방법을 알려 줌

STEP 3

③ 확인한다

실제로 열어 보고
원하는 대로 됐는지 판단

STEP 4

④ 고친다

부족한 점을 다시 말로 요청
→ ①로 돌아감

초보자가 가장 많이 하는 실수

- 한 번에 완벽한 것을 요구한다 → 원하는 것과 다른 결과가 나오고, 어디를 고칠지 모르게 됩니다.
- 확인하지 않고 계속 요청만 한다 → 오류가 쌓여 나중에 되돌리기 어려워집니다.
- 정답: 작게 만들고(①~②), 반드시 눈으로 확인하고(③), 한 번에 하나씩 고칩니다(④).

AI를 '오늘 입사한 신입사원'이라고 생각하세요

AI가 알고 있는 것

- 프로그램을 만드는 방법 (매우 뛰어남)
- 일반적인 업무 서식과 관행
- 요청하면 몇 번이든 다시 하는 인내심

AI가 전혀 모르는 것

- 우리 조직의 기준과 규칙
- 누가 무엇을 언제 쓰는지
- "알아서 잘"이 무엇을 의미하는지

그래서 이렇게 말해야 합니다

- 신입에게 지시하듯 — 목적 / 사용자 / 입력 / 출력 / 제약을 빠짐없이 알려 줍니다.
- "알아서 잘 만들어 줘"는 신입에게 "알아서 처리하세요"라고 말하는 것과 같습니다.
- 다만 신입과 달리, 잘못 만들어도 몇 번이든 다시 시킬 수 있습니다. 부담 없이 시도하세요.

좋은 요청의 5원칙

1

목적 — 무엇을 위한 것인가

"진행 중인 업무의 상태를 한눈에 보기 위한 화면"

2

사용자 — 누가 쓰는가

"담당자 1명이 사용. 컴퓨터에 익숙하지 않음"

3

입력과 출력 — 무엇을 넣고 무엇을 받는가

"엑셀 파일을 올리면 → 상태별 건수와 목록을 보여 줌"

4

화면 — 어떻게 생겼으면 하는가

"위쪽에 요약 숫자 4개, 아래쪽에 목록. 지연 건은 빨간색"

5

제약 — 지켜야 할 조건

"설치 없이 브라우저에서 열리게. 데이터는 내 PC에만 저장"

외우기 어렵다면 — "누가 / 무엇을 넣고 / 무엇을 얻고 / 어떻게 보이고 / 무엇을 하지 말 것"

EXAMPLE

같은 업무, 다른 요청 — 결과가 이렇게 갑니다

× 이렇게 말하면

"업무 관리 프로그램 만들어 줘"

결과

- AI가 임의로 기능을 상상해서 만들
- 우리가 일하는 방식과 전혀 다름
- 어디를 고쳐야 할지 판단하기 어려움
- 결국 처음부터 다시 요청하게 됨

○ 이렇게 말하면

"팀에서 진행 중인 업무 30건의 상태를 보는 화면을 만들어 줘. 엑셀(업무명·담당자·마감일·상태)을 올리면 상태별 건수를 요약해 표로 보여 주고, 마감이 3일 이내면 빨강게 표시해 줘. 브라우저에서 바로 열 수 있게."

결과

- 의도한 화면이 한 번에 나옴
- 기준이 명확해 결과를 검증할 수 있음
- "3일 → 5일로 바꿔 줘"처럼 수정이 쉬움
- 동료에게 설명하기도 쉬움

요청 전에 채우는 '한 장 기획서'

이 표만 채우면 그대로 붙여넣어도 훌륭한 요청이 됩니다

항목	질문	작성 예시
도구 이름	뭐라고 부를까?	업무 진행 현황판
목적	어떤 불편을 없애나?	지금 뭐가 어디까지 됐는지 매번 묻는 일을 없앤다
사용자	누가, 얼마나 자주?	담당자 1명, 매일 아침 확인
입력	무엇을 넣나?	업무명·담당자·마감일·상태
출력	무엇을 보고 싶나?	상태별 건수, 마감 임박 목록
화면	어떻게 보이길 원하나?	상단 요약 카드 4개 + 하단 표, 지연 건은 빨강
제약	지켜야 할 조건은?	샘플 데이터만 사용, 내 PC에서만 실행

이 표를 채우는 데 10분이면 충분합니다. 그리고 그 10분이 결과물의 품질을 결정합니다.

그대로 쓰는 요청 문장 3종

① 새로 만들 때

[누가] 가 사용할 [무엇을 하는] 화면을 만들어 줘.

[입력]을 넣으면 [출력]이 보이면 좋겠어. 화면은 [배치 설명]. [제약 조건]은 꼭 지켜 줘.

② 고칠 때

지금 화면에서 [어느 부분]이 [어떻게] 되어 있는데, [어떻게] 바꿔 줘. 나머지는 그대로 두고.

③ 오류가 났을 때

[무엇을] 했더니 아래 오류가 났어. 원인을 찾아서 고쳐 줘.

(오류 메시지를 그대로 복사해서 붙여넣기)

SKILL

수정 요청의 기술 4가지

만드는 것보다 고치는 과정이 훨씬 쉽니다

1. 한 번에 하나씩

"글씨 크게, 색 바꾸고, 검색도 넣고" → 하나만 요청하고 확인한 뒤 다음으로.

2. 위치를 특정한다

"버튼"이 아니라 "오른쪽 위 '저장' 버튼"처럼 어디인지 짚어 줍니다.

3. 결과를 묘사한다

"예쁘게"가 아니라 "글자 크기를 1.5배, 배경은 연한 회색으로".

4. 잘된 건 지킨다

"다른 기능은 건드리지 말고"를 붙이면 엉뚱한 변경을 막을 수 있습니다.

되돌리고 싶을 때 — "방금 변경한 내용을 취소하고 이전 상태로 돌려 줘"라고 말하면 됩니다.

오류가 나도 당황하지 마세요

오류는 실패가 아니라 대화의 재료입니다

STEP 1

① 멈추고 읽기

빨간 글씨가 나와도 컴퓨터는
고장 나지 않습니다

STEP 2

② 그대로 복사

오류 메시지 전체를 복사합니
다 (요약하지 말 것)

STEP 3

③ 상황과 함께 전달

"~을 눌렀더니 이 오류가 났어
"라고 붙여넣습니다

STEP 4

④ 확인 후 재시도

고쳐진 결과를 다시 실행해 보
니다

자주 나오는 상황

- 화면이 하얗게 비어 있음
- 버튼을 눌러도 반응이 없음
- 숫자 계산 결과가 이상함
- 파일을 못 찾겠다는 메시지

이렇게 말하면 됩니다

- "화면이 하얗게 나와. 원인 찾아서 고쳐 줘"
- "저장 버튼을 눌러도 목록에 안 생겨"
- "합계가 실제 값과 달라. 계산식 확인해 줘"
- "파일을 못 찾는데. 어디에 뒤야 해?"

만든 것을 실행하고, 보관하고, 공유하기

실행

어떻게 여나요?

작업 폴더에 생긴 파일을 더블클릭하면 브라우저에서 열립니다. 어디를 열어야 할지 모르면 "어떻게 실행해?"라고 물어보세요.

보관

어디에 있나요?

지정한 작업 폴더 안에 파일로 저장됩니다. 폴더 전체를 복사해 두면 그것이 곧 백업입니다.

공유

동료에게 주려면?

폴더를 통째로 압축해 전달하면 상대도 같은 방식으로 실행할 수 있습니다. 단, 데이터 포함 여부를 반드시 확인하세요.

공유 전 필수 확인 2가지

- ① 샘플이 아닌 실제 내부 자료가 파일에 들어 있지 않은가
- ② 규정상 외부로 나가면 안 되는 자료가 아닌가

오래 쓰는 사람들의 4가지 습관

1

작게 시작하고 자주 확인한다

기능 하나 → 확인 → 다음 기능. 한 번에 다 만들려 하지 않습니다.

2

잘 되던 시점을 복사해 둔다

폴더를 통째로 복사해 날짜를 붙여 두면 언제든지 되돌릴 수 있습니다.

3

요구사항을 파일로 남긴다

한 장 기획서를 작업 폴더에 저장해 두면 AI가 매번 참고합니다.

4

결과는 손으로 계산한다

합계·평균·건수 같은 숫자는 반드시 표본으로 직접 확인합니다.

특히 3번 — 작업 폴더에 '요구사항 메모'를 남겨 두는 습관이 유지보수 위험을 크게 줄여 줍니다.

바로 쓰는 요청 문장 모음

상황	이렇게 말하세요
처음 만들 때	[누가]가 쓸 [무엇을 하는] 화면을 만들어 줘. [입력]을 넣으면 [출력]이 보이게 해 줘.
기능 추가	지금 화면은 그대로 두고, [어디에] [무슨 기능]을 추가해 줘.
화면 수정	[어느 부분]의 [무엇]을 [어떻게] 바꿔 줘. 다른 곳은 건드리지 말고.
규칙 추가	[어떤 조건]일 때는 [어떻게] 동작하게 해 줘.
오류 해결	[무엇을] 했더니 아래 오류가 났어. 원인을 찾아 고쳐 줘. (오류 전문 붙여넣기)
되돌리기	방금 변경한 내용을 취소하고 이전 상태로 돌려 줘.
설명 요청	지금 만든 것이 어떻게 동작하는지 비전공자가 알아듣게 설명해 줘.
실행 방법	이 파일이 어디에 있고 어떻게 실행하는지 알려 줘.
검증 요청	계산이 정확한지 확인할 수 있게 예시 3건의 계산 과정을 보여 줘.

알아 두면 편한 용어 10개

몰라도 되지만, 알면 대화가 훨씬 빨라집니다

용어	쉬운 설명	용어	쉬운 설명
프롬프트	AI에게 전달하는 요청 문장	로컬	인터넷이 아닌 내 컴퓨터 안
생성형 AI	글·이미지·코드를 만들어 내는 AI	브라우저	웹 화면을 여는 프로그램
에이전트	파일 생성·실행까지 하는 AI	터미널	명령을 글자로 입력하는 창
환각	사실 아닌 것을 그럴듯하게 말함	폴더(디렉터리)	파일을 담아 두는 공간
프로토타입	완성 전 시험용 시제품	백업	잘 되던 시점을 복사해 두기

모르는 용어가 나오면 — "방금 말한 용어를 비전공자가 알아듣게 설명해 줘"라고 물어보면 됩니다.

제조 현장의 고민

- **제조 현장에서는 어떤 문제가 발생할까?**
 - 설비 고장
 - 생산 중단
 - 품질 불량
 - 수율 저하
 - 유지보수 비용 증가
- **대부분의 문제는 갑자기 발생하지 않는다.**
 - 고장 이전에는 작은 이상 신호가 존재한다.

이상징후(Anomaly Sign)란?

- 이상징후란 무엇인가?

→ 고장이 발생하기 전에 나타나는 비정상적인 변화

- 예시

- 진동 증가
- 온도 상승
- 전류 변화
- 압력 변동

- 이상징후 → 조기대응 → 고장 예방

제조 데이터란?

- 제조 현장에서 생성되는 데이터

- 설비 → 센서 데이터
- 공정 → 생산 데이터
- 품질 → 검사 데이터
- 운영 → 로그 데이터
- 환경 → 온도, 습도 등

시간	온도	압력	진동	전류
09:00	50	10.1	0.12	5.0
09:01	51	10.2	0.13	5.1
09:02	52	10.3	0.14	5.0

제조 데이터의 특징

1. 시계열 데이터

- 시간에 따라 변화

2. 다변량 데이터

- 여러 센서가 동시에 측정

3. 대용량 데이터

- 24시간 연속 생성

4. 노이즈 존재

- 센서 오차 발생 가능

제조 설비에서 수집되는 주요 데이터

센서 데이터

● 온도

● 압력

● 진동

● 전류

● 전압

생산 데이터

● 생산량

● Cycle Time

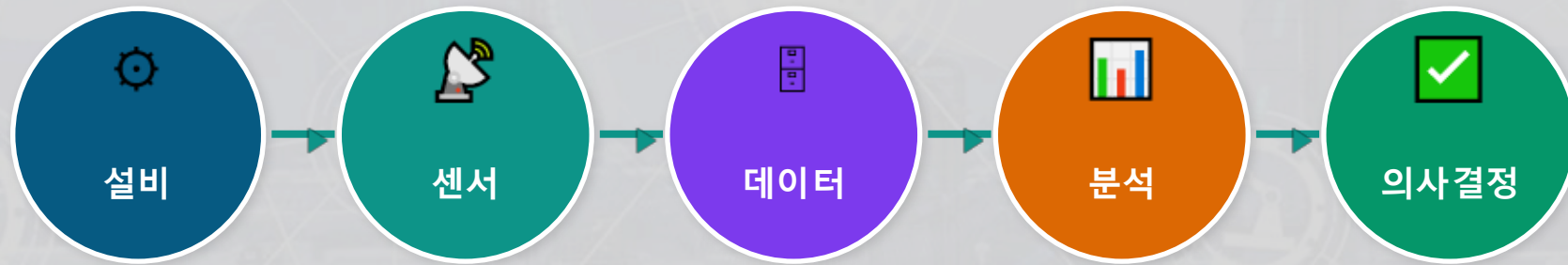
● 작업 속도

품질 데이터

● 불량률

● 검사 결과

스마트팩토리 데이터 흐름



이상(Anomaly)란?

- 정상 패턴과 다른 현상
- 상황에 따라 정의하기 나름
- 예시
 - 갑작스러운 온도 상승
 - 비정상적인 진동 증가
 - 생산 속도 급감
 - 불량률 증가

이상 징후(Anomaly Sign)란?

- 아직 고장은 아니지만 고장 가능성을 나타내는 신호
- 예시
 - 진동이 조금씩 증가
 - 온도가 지속적으로 상승
 - 전류 패턴 변화

이상과 고장의 차이

CATEGORY 구분	COLUMN A · WARNING 이상징후 Anomaly Sign	COLUMN B · CRITICAL 고장 Failure
ROW 01 · STATE 상태	WARNING 경고 단계	CRITICAL 실제 문제 발생
ROW 02 · PRODUCTION 생산 가능 여부	✓ 가능 설비는 작동 중, 모니터링 강화 단계	✗ 어려움 정지 또는 품질 불량 발생
ROW 03 · COST 대응 비용	■ 낮음 예방 정비 · 부품 교체 수준	■ 높음 설비 교체 · 라인 정지 · 납기 손실

제조 현장의 이상 유형

- Point Anomaly

특정 시점에 발생하는
단발성 이상 값

예시:

온도 50°C → 120°C
(순간 급등)

원인:

설비 과부하, 센서 오류,
일시적 공정 이상

Point Anomaly 예시 (온도)



제조 현장의 이상 유형

- Contextual Anomaly

DEFINITION

값 자체는 정상이지만,
상황(context)에 따라 이상

절대 임계값으로는 잡히지 않습니다. 계절·시간대·운영 모드 등 맥락 변수와 함께 봐야 비로소 이상으로 식별됩니다.

SAME VALUE

30°C – 겨울 정상 | 여름 비정상

EXAMPLE · 같은 값, 다른 맥락

CONTEXT A · WINTER

겨울철

30°C

정상

외기 0°C 대비 작동 온도로 적정 범위.

CONTEXT B · SUMMER

여름철

30°C

비정상

외기 35°C + 작동 중이면 추가 상승 필요.

제조 현장의 이상 유형

- Collective Anomaly

개별 데이터는 정상
연속된 패턴이 이상

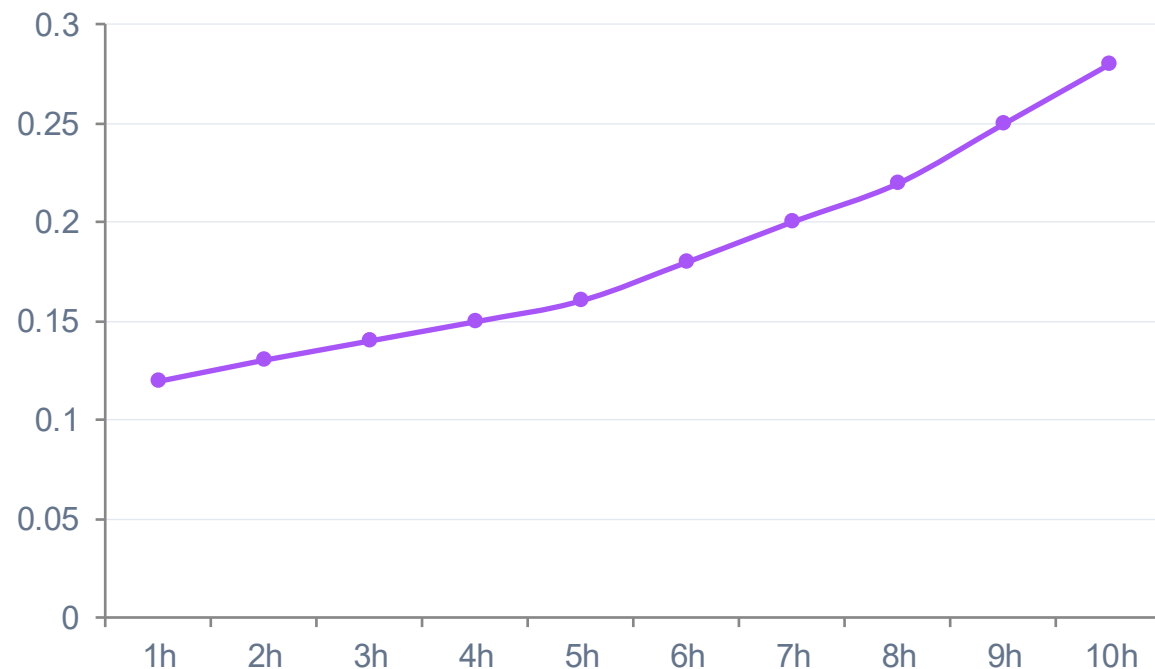
예시:

진동이 서서히 증가
(개별값은 정상 범위
지속적 상승 트렌드)

탐지 방법:

추세 분석, 이동 평균,
시계열 모델 활용

Collective Anomaly 예시 (진동 서서히 증가)

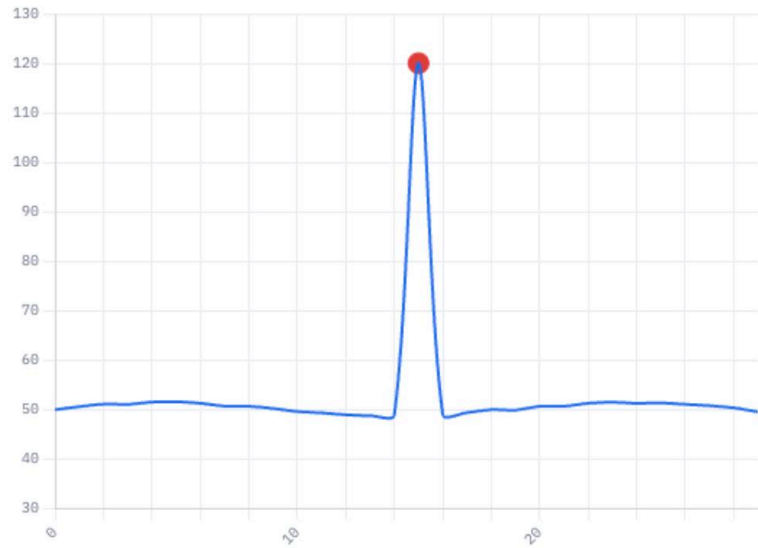


이상 유형 예시 그래프

TYPE 01

Point Anomaly

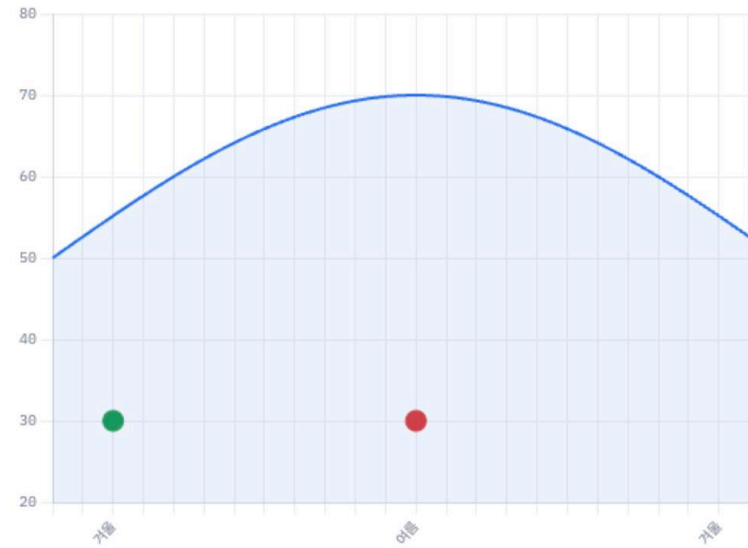
한 점이 튀다



TYPE 02

Contextual Anomaly

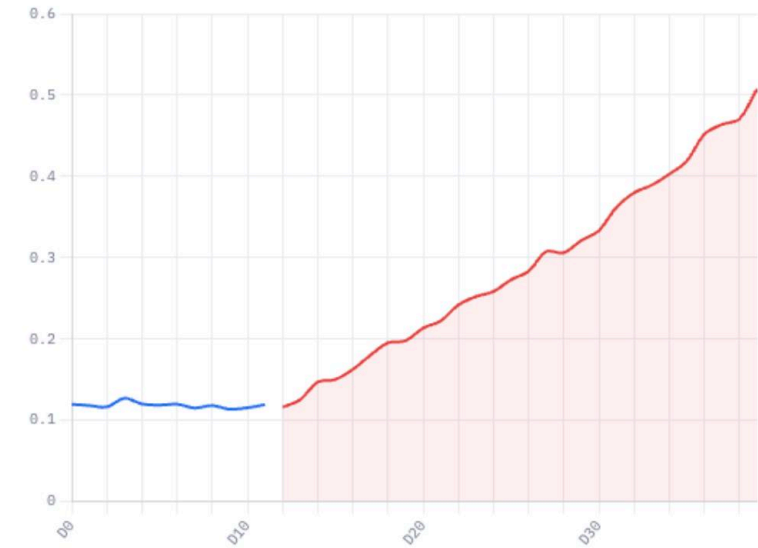
맥락이 다르다



TYPE 03

Collective Anomaly

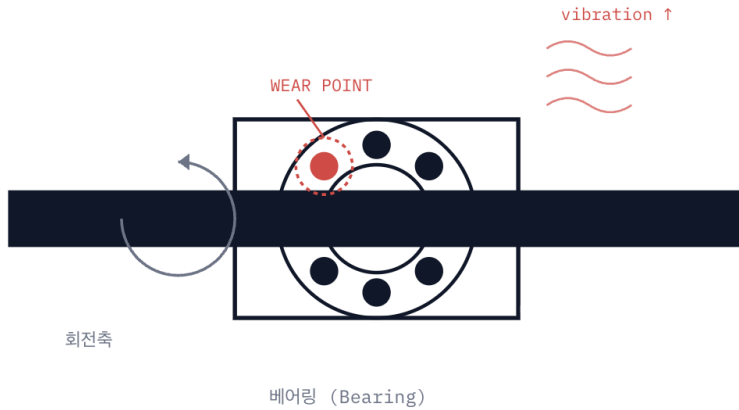
패턴이 이어진다



제조 현장의 대표 사례

베어링 마모 Bearing Wear

SCHEMATIC · 회전축 + 베어링



SYMPTOM · 증상

SIGNAL A

진동 증가 ↑

Vibration RMS rises

SIGNAL B

소음 증가 ↑

Audible noise rises

RESULT · 결과

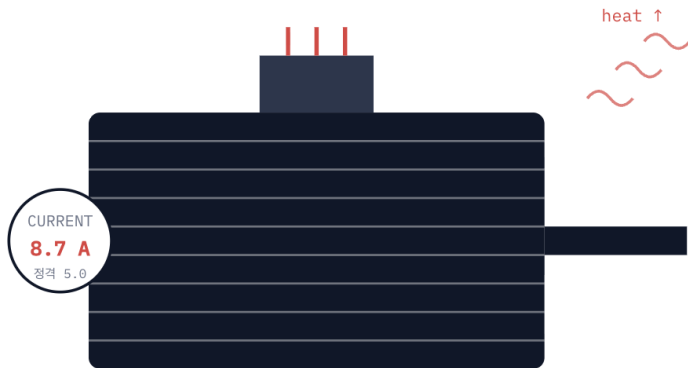
설비 정지

계획 외 정지 발생 →
라인 전체 가동 손실

제조 현장의 대표 사례

모터 이상 Motor Abnormality

SCHEMATIC · 모터 단자/하우징



유도 전동기 (Induction Motor)

SYMPTOM · 증상

SIGNAL A

전류 증가 ↑

Current draw rises beyond rated

SIGNAL B

온도 상승 ↑

Winding temperature rises

RESULT · 결과

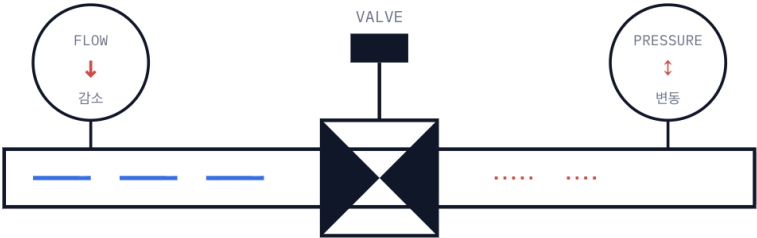
생산 중단

모터 소손 시 교체 리드타임 ↑ →
생산 일정 차질

제조 현장의 대표 사례

공정 이상 Process Deviation

SCHEMATIC · 배관 + 압력/유량



유체 공급 라인 (Fluid Line)

SYMPTOM · 증상

SIGNAL A

압력 변화 ⬆

Pressure fluctuates

SIGNAL B

유량 감소 ⬇

Flow rate drops

RESULT · 결과

품질 불량

공정 조건 이탈로 제품 사양 불일치 →
재작업 또는 폐기 비용 발생

실제 사례 – 반도체 제조

STEP 01 · TRIGGER



온도 센서 이상

캘리브레이션 드리프트로 실제 온도보다 낮게 측정



STEP 02 · DRIFT



공정 편차 증가

증착·식각 두께가 사양 범위를 벗어나기 시작



STEP 03 · IMPACT



수율 감소

한 로트 전체가 합격선을 통과하지 못함



STEP 04 · COST



막대한 비용 발생

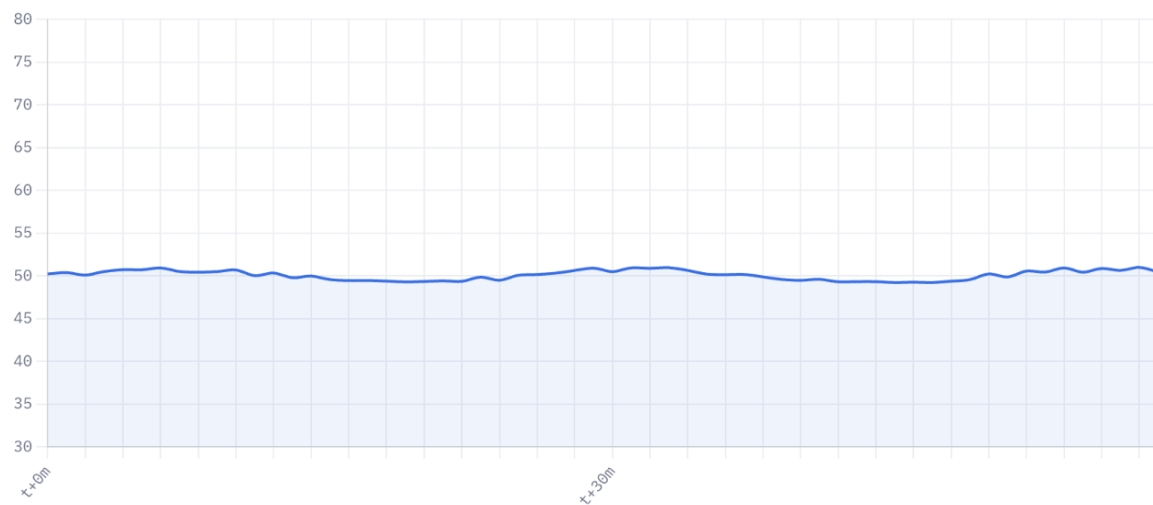
웨이퍼 한 장 수십만 원 × 수천 장

데이터로 보면 어떻게 보일까?

NORMAL WINDOW · 정상 구간

평균 50.2°C · σ 0.8

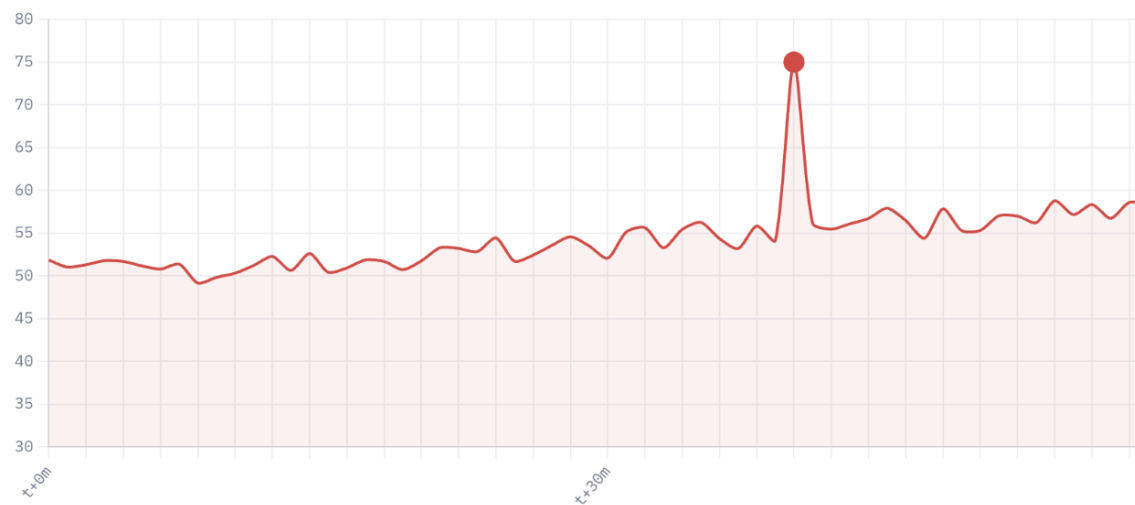
좁은 변동폭, 일정한 주기



ANOMALOUS WINDOW · 이상 구간

평균 58.4°C · σ 4.6

평균 상승 + 분산 확대 + 스파이크



이상 발생 전 패턴

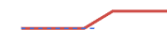
CORE THESIS

고장 직전에는,
종종 작은 변화가
먼저 발생합니다.

"갑자기" 고장난 것처럼 보여도 — 데이터를 되돌려 보면,
신호는 며칠 전부터 이미 존재했습니다.

EARLY SIGNALS · 세 가지 약한 신호

01 평균 변화 Mean shift
기준선이 서서히 위 또는 아래로 이동



02 분산 증가 Variance widening
변동폭이 점점 커지며 불안정해짐



03 주기 변화 Frequency change
사이클이 짧아지거나 길어짐



이상 탐지의 목표

01

BASELINE

정상

평소의 운전 패턴을 학습하고 기준선을 확보한다.



02

DETECT

이상징후 발견

기준선과의 편차·추세 변화를 자동으로 포착한다.



03

RESPOND

조기 대응

알림 → 점검 → 부품 교체로 골든타임을 활용한다.



04

PREVENT

고장 예방

→ 계획 외 정지 0, 품질 불량 최소화, 비용 절감

Noise vs Outlier vs Anomaly

- Noise (노이즈/잡음)
- Outlier (이상치)
- Anomaly (이상/이상현상/이상징후)

Noise

- 측정 과정에서 발생하는 작은 오차
- 예시: 50.0, 49.9, 50.1, 50.2, 49.8, 50.0
- 원인
 - 센서 오차
 - 통신 오차
 - 환경 변화
- 보통 특별한 의미 없다.

Noise

- 측정 과정에서 발생하는 작은 오차
- 예시: 50.0, 49.9, 50.1, 50.2, 49.8, 50.0
- 원인
 - 센서 오차
 - 통신 오차
 - 환경 변화
- 보통 특별한 의미 없다.

Noise

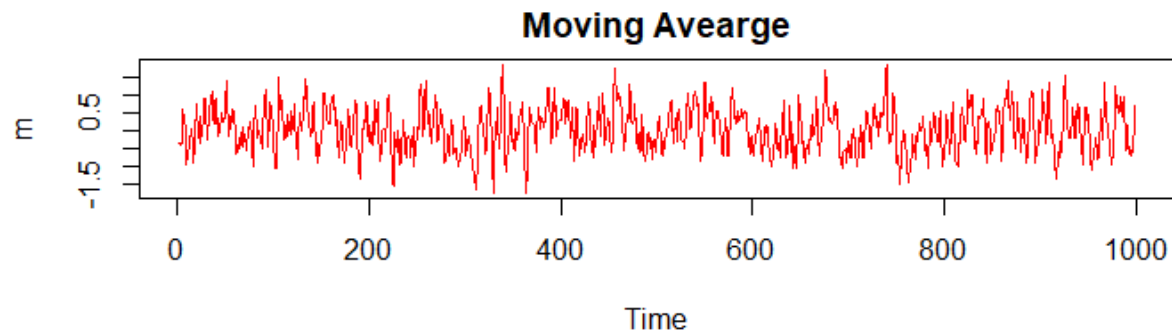
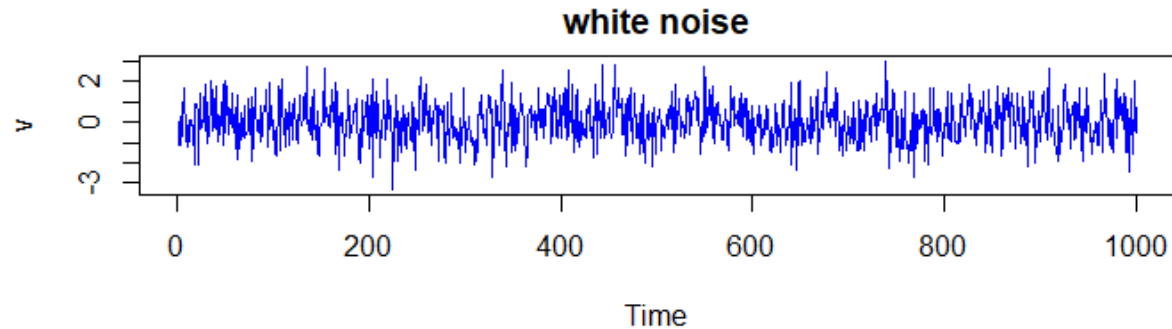
- 정상 신호 주변의 작은 변동
- 실제 온도: 50°C
 - ↓
- 측정값:
 - 49.9°C
 - 50.1°C
 - 50.2°C
 - 49.8°C
- 완벽하게 동일하게 측정되지는 않는다.

Noise가 생기는 원인

- 원인 1: 센서 자체의 한계, 모든 센서는 오차를 가진다.
- 원인 2: 전기적 간섭. 현장의 다른 장비로 인한 전자기 간섭이 센서 신호에 영향을 줄 수 있다.
- 원인 3: 환경 변화. 온도, 진동, 먼지 등.
- 원인 4: 디지털 변환 과정.
- 원인 5: 통신 과정. 전송 과정에서도 일부 값이 왜곡될 수 있다.

Noise는 어떻게 처리할까?

- Noise는 Moving Average, Filtering, Smoothing 등의 방법을 사용하여 처리한다.
- 가능한 줄이거나 제거한다.
- 노이즈는 분석을 방해하기 때문이다.



Outlier와 Anomaly는 같은 것일까?

많은 사람들이 $\text{Outlier} = \text{Anomaly}$ 라고 생각한다

하지만

두 개는 다른 개념이다

Outlier (이상치)

데이터 관점의 개념

다른 데이터와 크게 다른 값

50

51

49

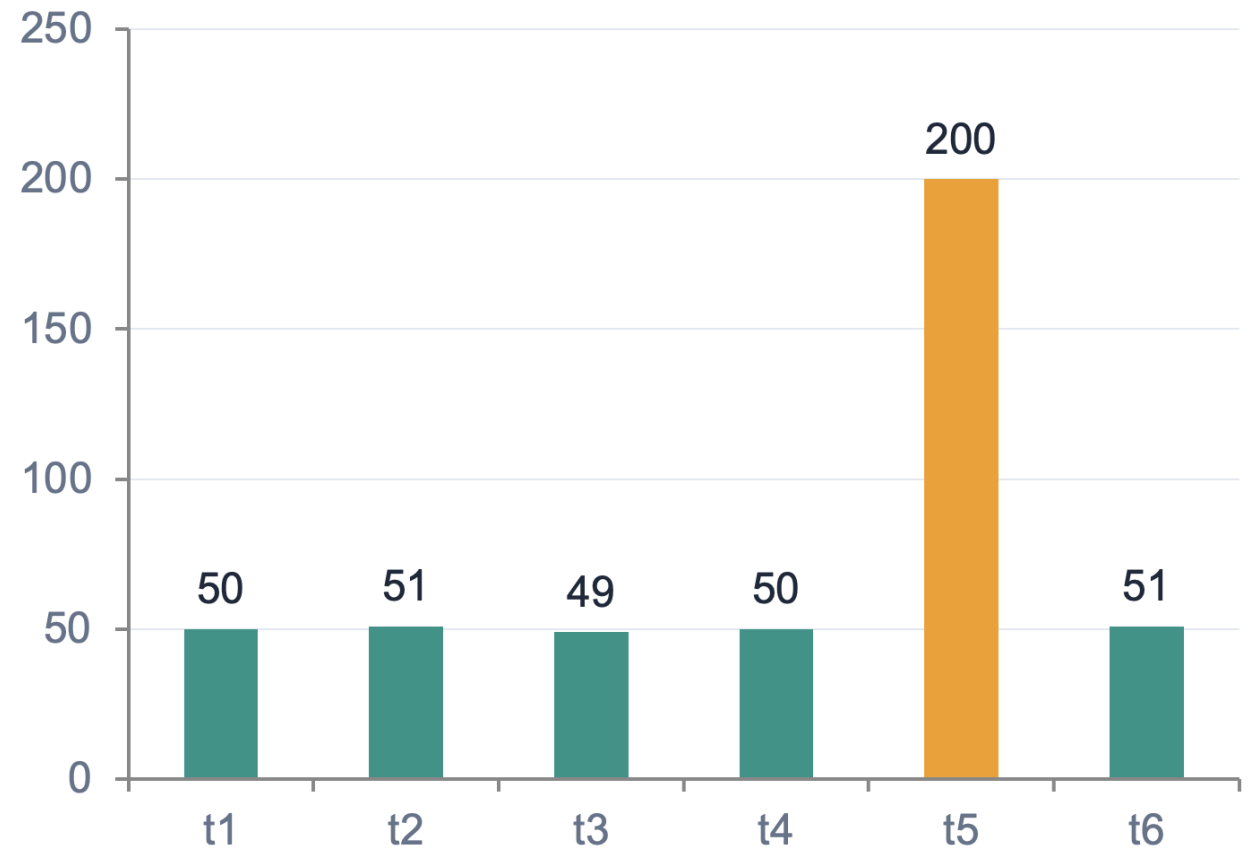
50

200

← Outlier!

51

데이터 중 t5가 Outlier



Outlier (이상치)의 정의

Outlier

통계적으로 다른 값들과 크게 다른 데이터 포인트

질문

"이 값이 다른 값들과 다른가?"

관심 대상

값 (Value)

탐지 방법

IQR, Z-score, 박스플롯 등

Anomaly (이상 현상)

현상 관점의 개념

실제로 의미 있는
비정상 상태

⚙ 베어링 마모

🔥 모터 과열

⚠ 공정 이상

✖ 설비 고장

Anomaly



Good



Good



Good



Anomaly의 정의

Anomaly

실제 설비 또는 공정에 문제가 있는 상태

질문

"실제로 문제가
발생하고 있는가?"

관심 대상

현상 (Phenomenon)

탐지 방법

AI/ML 모델,
도메인 지식 활용

Outlier O, Anomaly X — 센서 오류

데이터

50

51

49

50

200

51

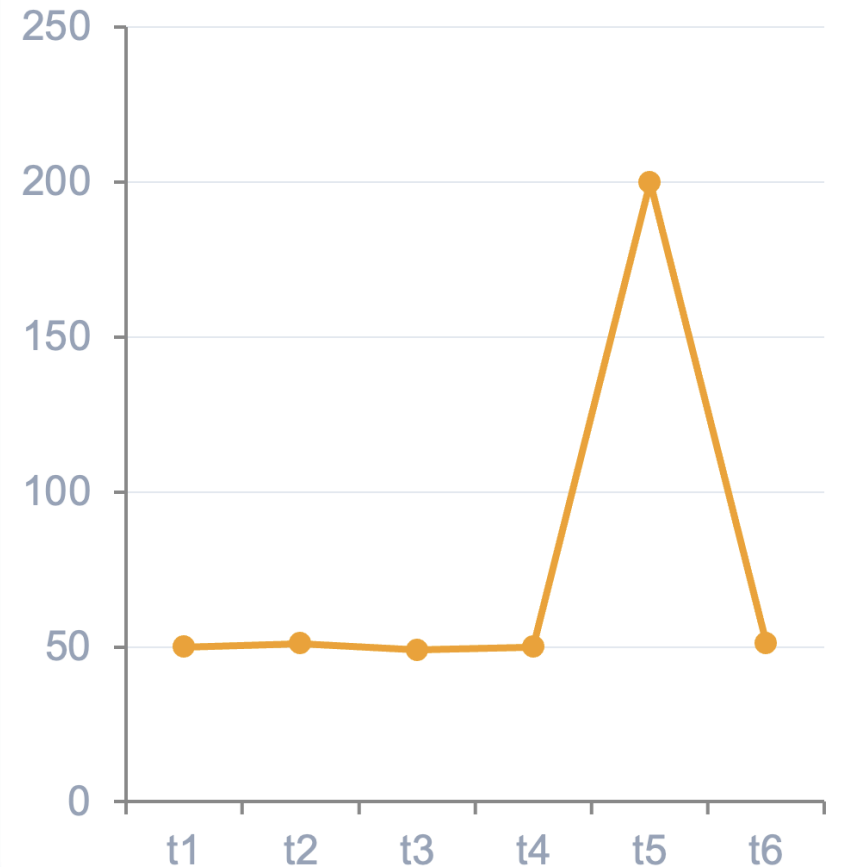
원인 분석

t5에서 200이 찍혔지만
실제 설비는 정상 운영 중.
→ 센서 오류로 판명

✓ Outlier

✗ Anomaly

Outlier O / Anomaly X



Outlier O, Anomaly O — 모터 과열

데이터

50

51

49

50

90

95

100

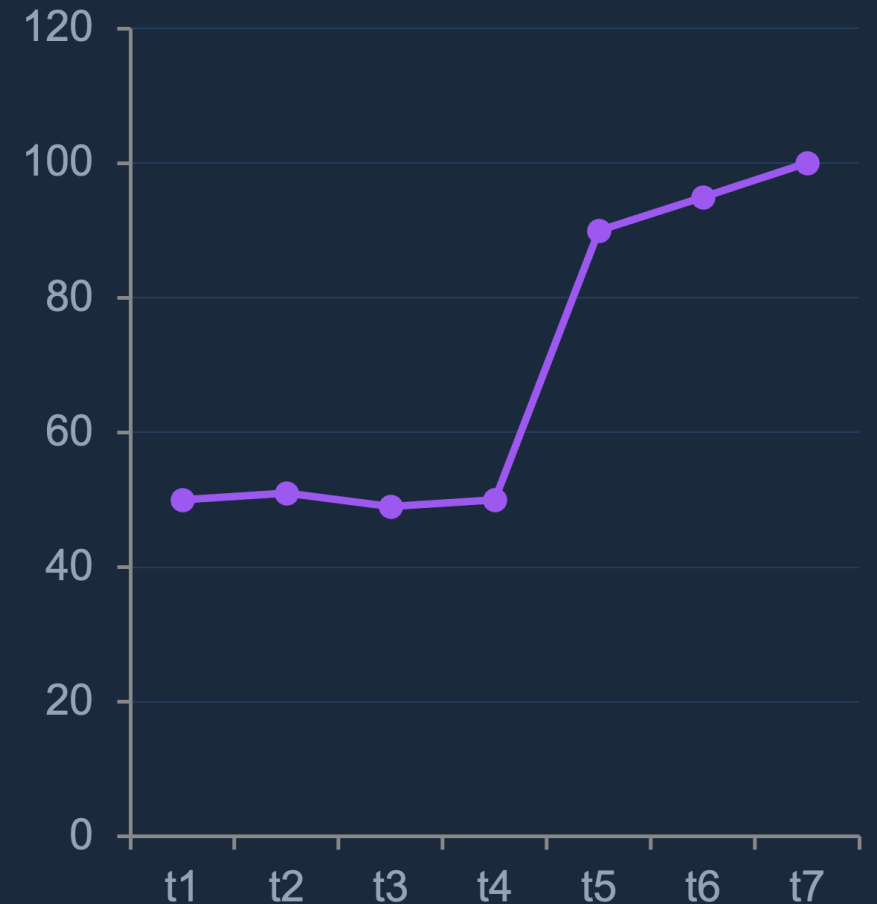
원인 분석

t5~t7에서 온도 급등.
→ 모터 과열로 실제 설비 이상 확인

✓ Outlier

✓ Anomaly

Outlier O / Anomaly O



Outlier X, Anomaly O — 베어링 마모

데이터

0.1

0.11

0.12

0.13

0.14

0.15

0.16

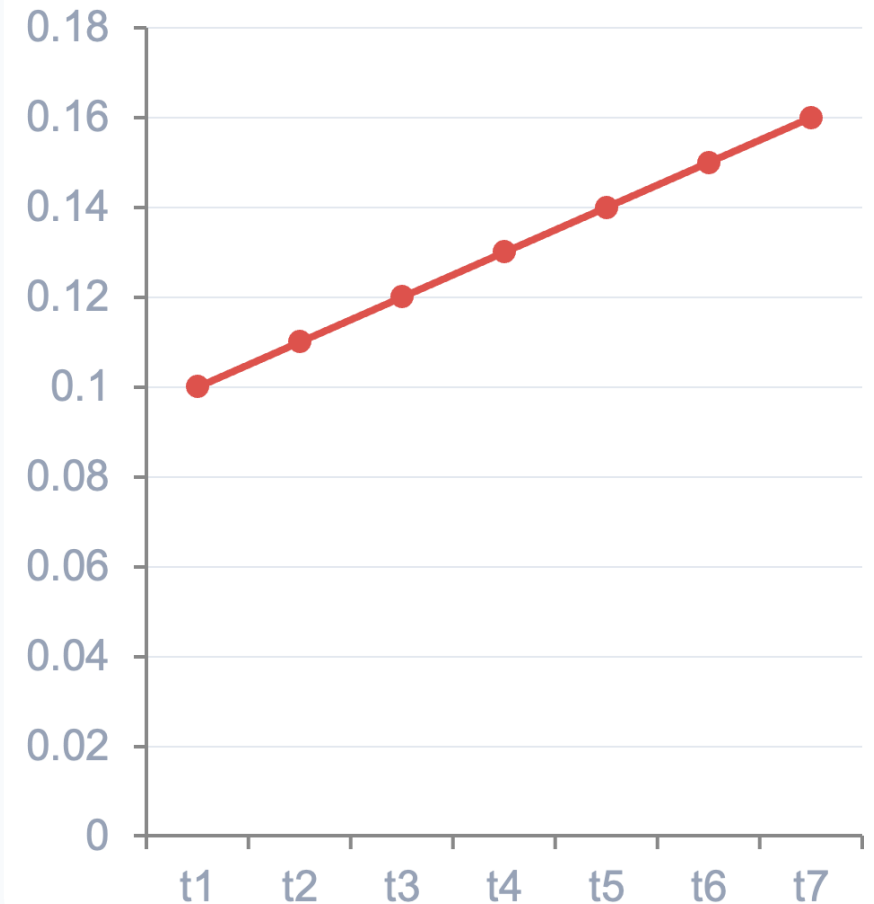
원인 분석

각 값은 모두 정상 범위.
하지만 진동이 지속적으로
증가 → 베어링 마모

✗ Outlier

✓ Anomaly

Outlier X / Anomaly O



왜 Anomaly인데 Outlier가 아닐까?

각 시점만 보면...

0.10 → 정상 범위

0.11 → 정상 범위

0.12 → 정상 범위

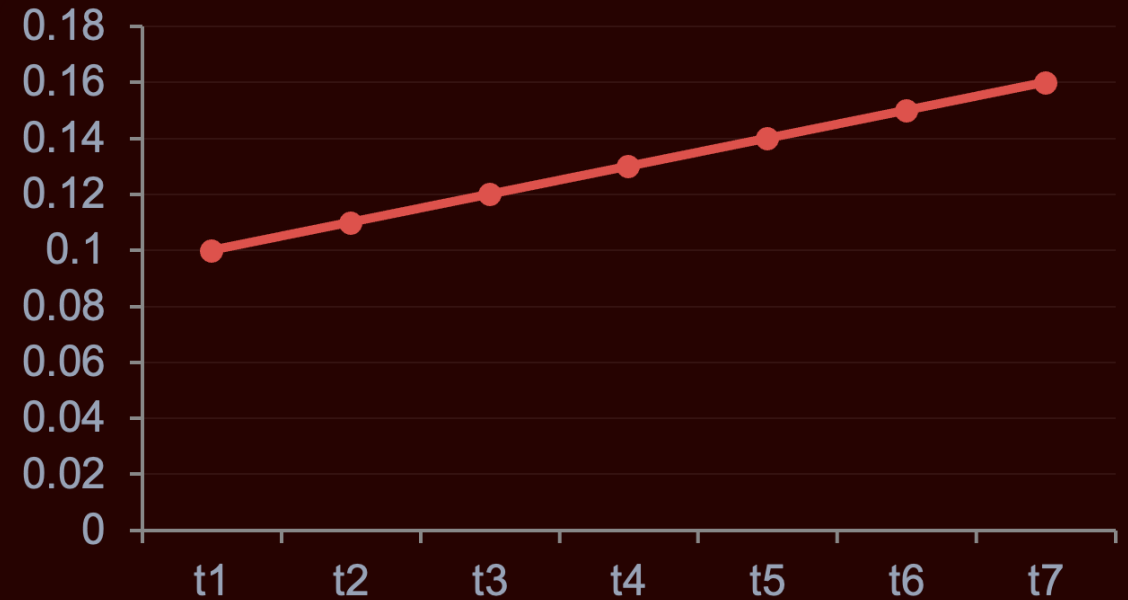
0.13 → 정상 범위

0.14 → 정상 범위

0.15 → 정상 범위

0.16 → 정상 범위

패턴 전체를 보면...



⚠ 지속적인 증가 = Anomaly!

Outlier vs Anomaly

구분	Outlier	Anomaly
관점	데이터 (Data)	현상 (Phenomenon)
의미	통계적으로 특이한 값	실제 이상 상태
질문	값이 이상한가?	실제 문제가 있는가?

제조 데이터는 어떻게 저장될까?

- 제조 현장의 데이터
 - 센서와 설비에서는 지속적으로 데이터가 생성된다

- 예시:

시간	온도	압력	진동
09:00	35.2	1.5	0.02
09:01	35.3	1.6	0.03
09:02	35.5	1.7	0.02

- 일반적인 저장 형태
 - CSV
 - Excel
- 제조 데이터는 대부분 표 형태로 저장된다

행(Row)과 열(Column)

시간	온도	압력	진동
09:00	35.2	1.5	0.02
09:01	35.3	1.6	0.03
09:02	35.5	1.7	0.02

- 행: 하나의 관측값
 - e.g., 09:00의 공정 상태
- 열: 하나의 변수(Variable)
 - e.g., 온도, 압력, 진동

제조 데이터의 변수들

시간	온도	압력	전류	품질
09:00	35.2	1.5	0.02	정상
09:01	35.3	1.6	0.03	정상
09:02	35.5	1.7	0.02	불량

- 공정 변수(Process Variables)
 - 온도, 압력, 속도, 전류
- 품질 변수(Quality Variables)
 - 정상/불량, 제품 치수, 강도
- 공정 변수는 원인, 품질 변수는 결과인 경우가 많다

데이터 타입 이해

시간	온도	압력	전류	품질
09:00	35.2	1.5	0.02	정상
09:01	35.3	1.6	0.03	정상
09:02	35.5	1.7	0.02	불량

- 숫자형 데이터
 - e.g., 온도, 압력, 속도
- 범주형 데이터
 - e.g., 정상, 불량
- 날짜/시간 데이터
 - e.g., 2025-05-01 09:00

시계열(Time-Series) 데이터

시간	온도	압력	전류	품질
09:00	35.2	1.5	0.02	정상
09:01	35.3	1.6	0.03	정상
09:02	35.5	1.7	0.02	불량

- 많은 제조 데이터는 시계열 데이터
 - 시간 순서가 존재
- 현재 상태는 과거 사태의 영향 받는다



Feature와 Target

- Feature (입력 변수)
 - 온도
 - 압력
 - 전류
- Target (예측 대상)
 - 불량 여부
 - 생산량
 - 설비 고장
- Feature를 이용하여 Target을 설명하거나 예측한다

왜 통계가 필요할까

제조 데이터의 문제

- 하루 동안 측정된 온도: 35.1, 34.8, 35.4, 35.2, 34.9, 35.0, ...
- 수천 개의 숫자가 존재
- 질문: 이 데이터를 한 문장으로 요약할 수 있을까?
- 해결방법: 통계량(Statistics)
 - 데이터를 몇 개의 숫자로 요약
 - e.g., 평균, 중앙값, 최빈값, 최소값, 최대값, 범위, 분산, 표준편차

평균(Mean)

- 데이터의 대표값

- 공식

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

- 예시

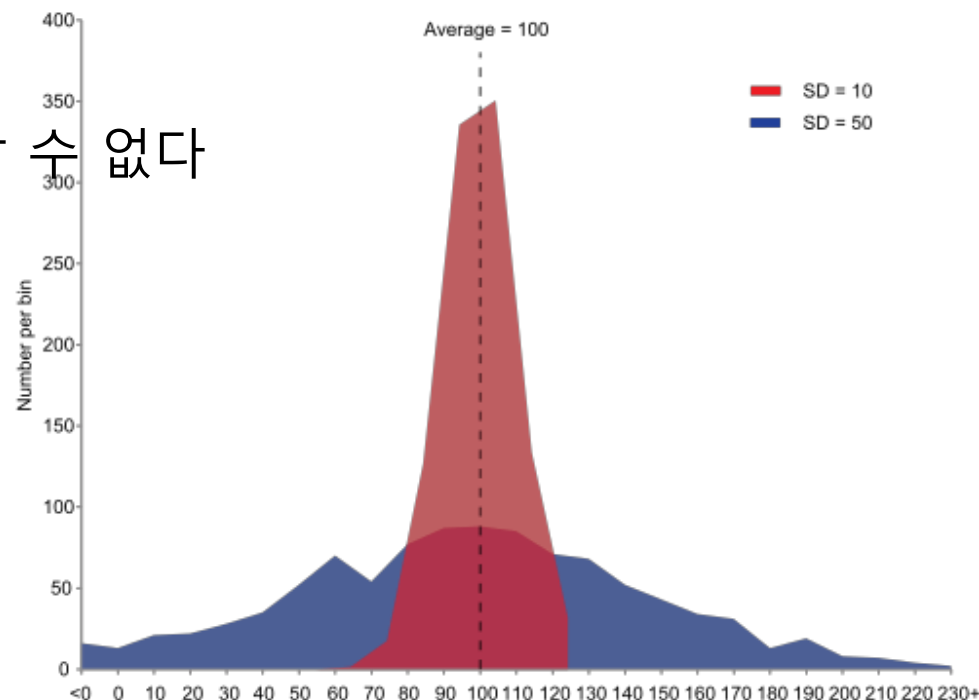
- 온도 데이터: 30, 35, 40, 45, 50

- 평균: $(30+35+40+45+50)/5 = 40$

- 전반적인 수준을 나타냄

평균의 한계

- 데이터 A: 38, 39, 40, 41, 32
 - 평균 = 40
- 데이터 B: 10, 20, 40, 60, 70
 - 평균 = 40
- 평균은 동일하지만 데이터 특성은 완전히 다름
- 평균은 중심만 알려줄 뿐, 데이터의 퍼짐 정도는 알 수 없다



분산(Variance)

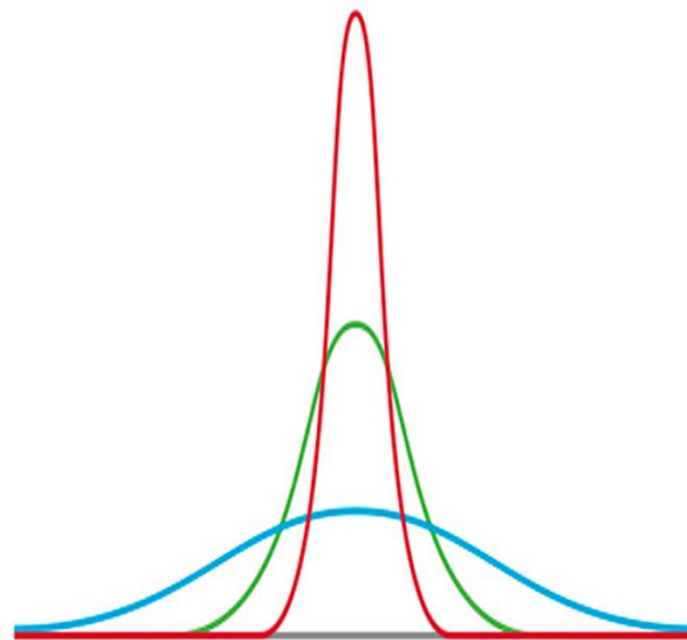
- 데이터가 평균으로부터 얼마나 퍼져 있는지를 나타내는 지표

- 공식

$$\text{Var}(X) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

- 직관적 의미

- 분산이 작음 $\rightarrow$ 데이터가 평균 주변에 모여 있음
- 분산이 큼 $\rightarrow$ 데이터가 넓게 퍼져 있음



표준편차(Standard Deviation)

- 데이터가 평균으로부터 얼마나 퍼져 있는지를 나타내는 지표
- 분산의 문제점: 분산은 차이를 제곱해서 계산
- 예시
 - 온도 단위 : °C
 - 분산 단위 : °C²
- 직관적으로 이해하기 어려움
- 해결 방법: 분산에 제곱근을 적용

$$\text{표준편차} = \sqrt{\text{분산}}$$

- 공식

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}$$

평균, 분산, 표준편차 비교

통계량	의미
평균	데이터의 중심
분산	데이터의 퍼짐 정도
표준편차	평균으로부터의 평균적인 거리

- 평균 → 공정 수준
- 표준편차 → 공정 안정성
- 평균은 어디 근처에 있는가를 알려주고, 표준편차는 얼마나 흔들리는가를 알려준다

데이터 전처리란?

- 데이터 전처리(Data Preprocessing): 원시 데이터를 분석 가능한 형태로 변환하는 과정
- 제조 현장의 실제 데이터:

SECOM Data Head:																					
	0	1	2	3	4	5	6	7	8	9	...	580	581	582	583	584	585	586	587	588	589
0	3030.93	2564.00	2187.7333	1411.1265	1.3602	100.0	97.6133	0.1242	1.5005	0.0162	...	NaN	NaN	0.5005	0.0118	0.0035	2.3630	NaN	NaN	NaN	NaN
1	3095.78	2465.14	2230.4222	1463.6606	0.8294	100.0	102.3433	0.1247	1.4966	-0.0005	...	0.0060	208.2045	0.5019	0.0223	0.0055	4.4447	0.0096	0.0201	0.0060	208.2045
2	2932.61	2559.94	2186.4111	1698.0172	1.5102	100.0	95.4878	0.1241	1.4436	0.0041	...	0.0148	82.8602	0.4958	0.0157	0.0039	3.1745	0.0584	0.0484	0.0148	82.8602
3	2988.72	2479.90	2199.0333	909.7926	1.3204	100.0	104.2367	0.1217	1.4882	-0.0124	...	0.0044	73.8432	0.4990	0.0103	0.0025	2.0544	0.0202	0.0149	0.0044	73.8432
4	3032.24	2502.87	2233.3667	1326.5200	1.5334	100.0	100.3967	0.1235	1.5031	-0.0031	...	NaN	NaN	0.4800	0.4766	0.1045	99.3032	0.0202	0.0149	0.0044	73.8432
5 rows × 590 columns																					

- 바로 분석 가능할까?
- 먼저 해야 할 일: 결측치 처리, 이상치 처리, 데이터 정제

왜 전처리가 필요할까?

- 실제 제조 데이터는
 - 누락됨
 - 잘못 측정됨
 - 노이즈가 존재함
 - 불필요한 변수가 존재함
- 예시
 - 온도 데이터: 35, 36, 37, 999, 38, NA
 - 이 데이터를 그대로 분석하면 잘못된 결과 가능
- 데이터 품질이 분석 결과를 결정한다

Garbage In, Garbage Out

- GIGO 원칙
 - Garbage In → Garbage Out
- 의미
 - 잘못된 데이터 → 잘못된 분석 결과
- 예시
 - 센서 오류: 35, 36, 999, 37, 38
 - 평균 계산: 289
 - 실제 공정 상태를 전혀 반영하지 못함
- 좋은 모델보다 좋은 데이터가 먼저이다

제조 데이터에서 흔히 발생하는 문제

- 결측값
 - 센서 고장, 통신 오류, 측정 실패
- 의미 없는 센서
 - 항상 같은 값
- 노이즈
 - 작은 측정 오차
- 이상치
 - 비정상적으로 크거나 작은 값
- 제조 데이터는 대부분 전처리가 필요하다

데이터 전처리 과정

- 일반적인 분석 절차
 - 데이터 수집 → 데이터 전처리 → 데이터 분석 → 의사결정
- 전처리 단계
 - 결측치 처리
 - 이상치 처리
 - 변수 선택
 - 데이터 정제
- 전처리는 데이터 분석의 필수 단계이다

데이터 전처리 예시

온도	→	온도
35		35
36		36
NA		36.5
37		37
500		제거

- 신뢰할 수 있는 데이터 확보
 - 데이터 분석을 위한 머신러닝 모델이 입력으로 받아들일 수 있는 형태로 바꿈
 - 전처리 후 결과가 실제 공정을 더 잘 반영
- 전처리는 데이터를 깨끗하게 만드는 과정

결측치 처리 전략

- 결측치 → 통계량 왜곡, 패턴 분석 어려움, AI 모델 학습 불가, 이상 현상 분석 실패
- 결측치는 단순한 빈칸이 아니라 정보의 손실
- 가장 먼저 해야 할 일
 - 결측치의 양을 확인

변수	결측 비율
온도	1%
압력	2%
진동	45%

- 결측치 처리 전에 먼저 현황을 파악해야 한다

결측치 처리 방법 1: 행 삭제

- 결측이 포함된 행 제거

온도	압력
35	1.5
NA	1.6
37	1.7



온도	압력
35	1.5
37	1.7

- 장점
 - 매우 간단
 - 구현 쉬움
- 단점
 - 데이터 손실 발생
- 사용 시점: 결측 비율이 매우 낮을 때

결측치 처리 방법 2: 평균 대체

- 결측값을 평균으로 대체
- 예시

온도	압력
40	1.2
42	NA
NA	1.8
44	6.0



온도	압력
40	1.2
42	3.0
42	1.8
44	6.0

- 빠르고 구현이 쉽다

결측치 처리 방법 3: 중앙값 대체

- 결측값을 중앙값으로 대체
- 예시

온도	압력
40	1.2
42	NA
NA	1.8
44	6.0



온도	압력
40	1.2
42	1.8
42	1.8
44	6.0

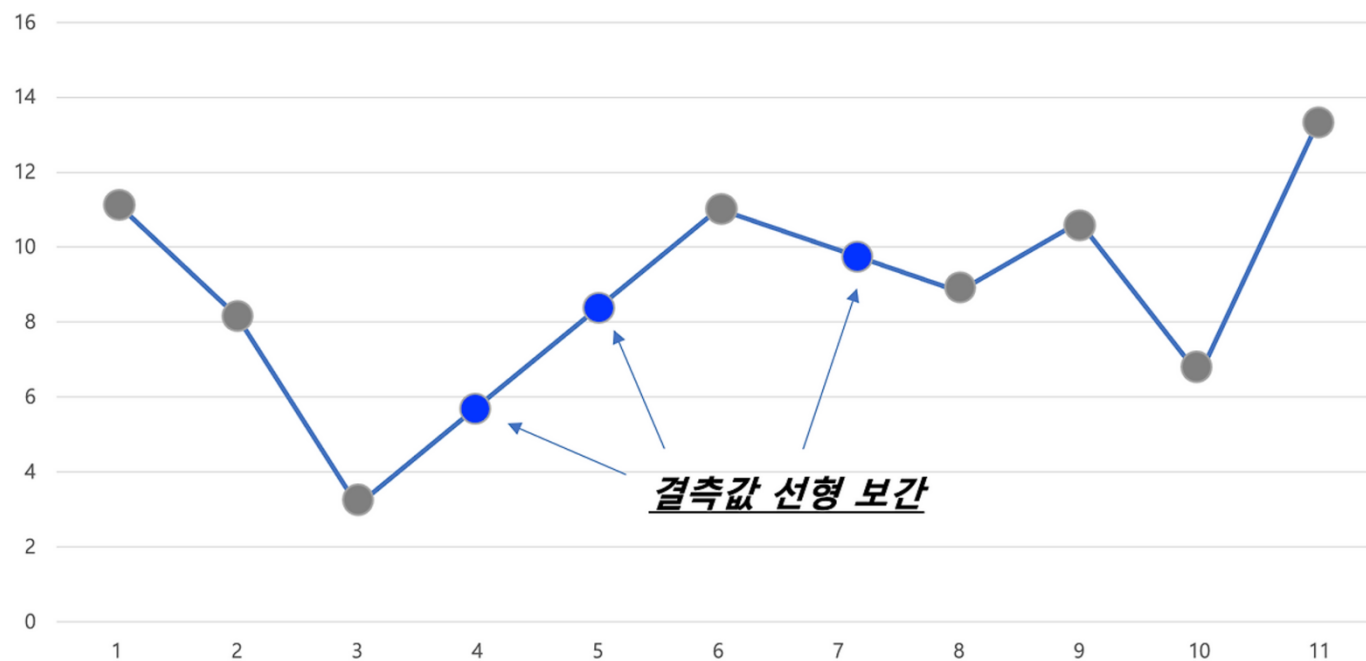
- 이상치에 강인하다.

결측치 처리 방법 4: 선형 보간

- 제조 시계열 데이터에서 자주 사용

lot_id	cell_id	param_id	timestamp_id	measure_val
1	1	1	1	10.235654250405200
1	1	1	2	6.420217266962500
1	1	1	3	
1	1	1	4	16.278196867069900
1	1	1	5	11.043981185063200
1	1	1	6	8.173617377009170
1	1	1	7	
1	1	1	8	9.405176524991060
1	1	1	9	8.048817274030900

⋮ ⋮ ⋮ ⋮ ⋮

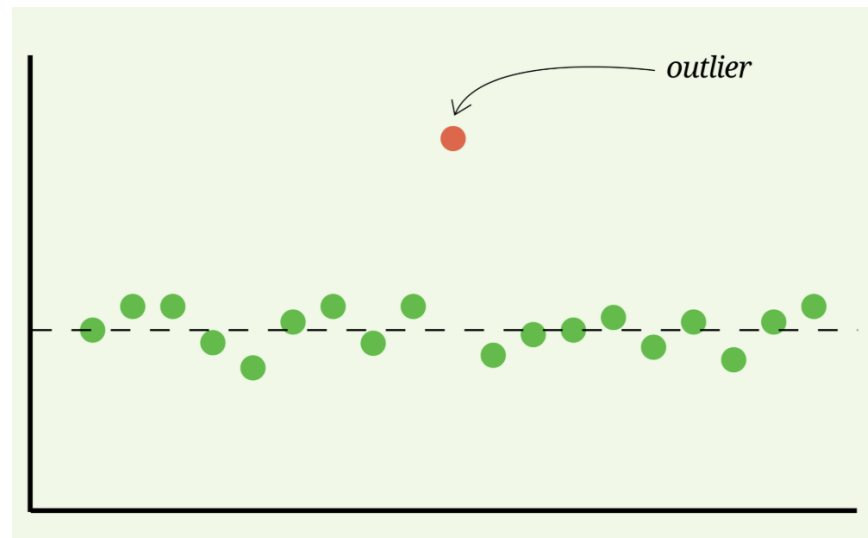


결측치 처리 방법 5: kNN 대치

- 비슷한 데이터 활용
- 결측이 포함된 각 데이터와 비슷한 k개의 공정 상태를 찾음
- k개의 공정 상태로 결측값 추정
- 장점
 - 단순 평균 대치보다 더 사실적인 대치가 가능하다
- 단점
 - 데이터 크기가 클 경우 계산량이 커질 수 있다

이상치

- 이상치(Outlier)란?
 - 다른 데이터와 비교했을 때 현저히 크거나 작은 값
- 예시
 - 온도: 48, 49, 50, 51, 52, 300
- 질문: 300은 정상 데이터일까?
- 센서 오류일 수도 있고 실제 공정 이상일 수도 있음



왜 이상치가 문제일까?

- 실제 공정 상태를 제대로 반영하지 못하여 데이터 분석을 방해함
- 예시: 평균 왜곡
 - 정상 데이터: 48, 49, 50, 51, 52
→ 평균 = 50
 - 이상치 포함: 48, 49, 50, 51, 52, 300
→ 평균 = 92
- 머신러닝 성능 저하
 - 회귀 모델, 통계 분석, 예측 모델에 큰 영향을 줄 수 있음

이상치 제거 전에 확인해야 할 것

- Case 1. 센서 오류

50 → 51 → 300 → 50 → 49

- 제거 후보

- Case 2. 실제 공정 이상

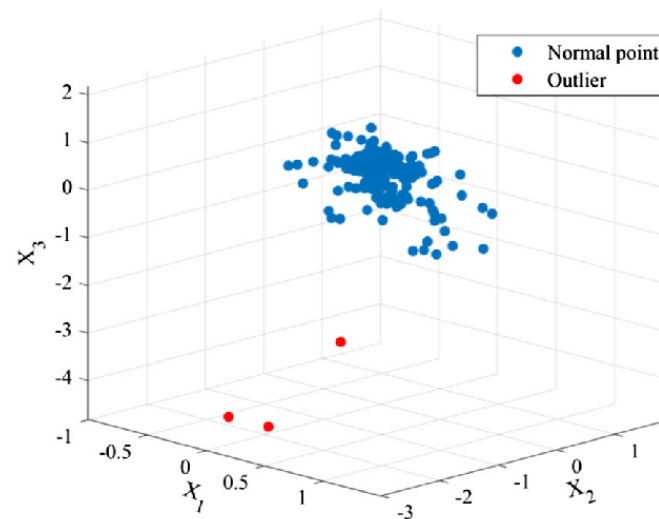
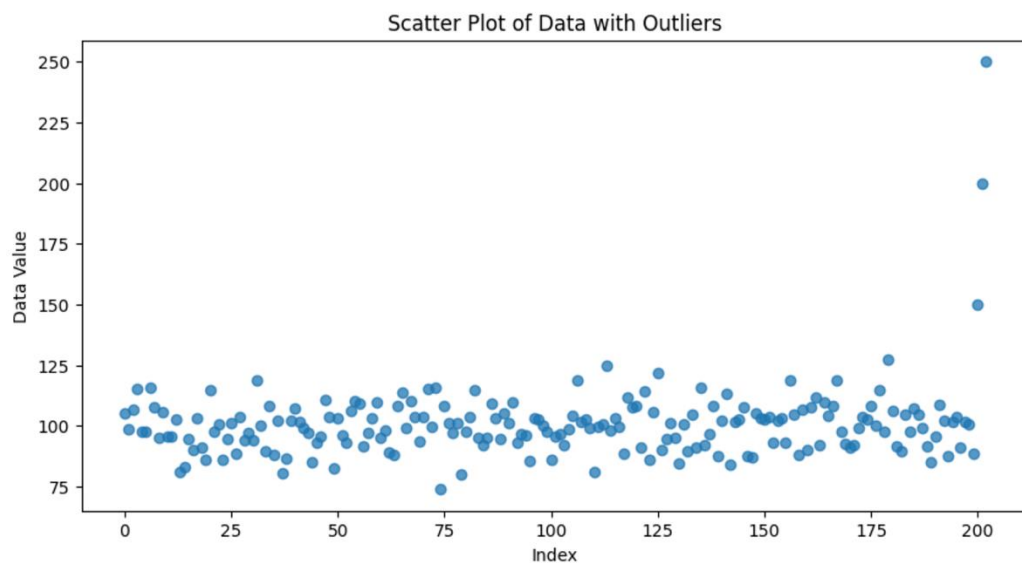
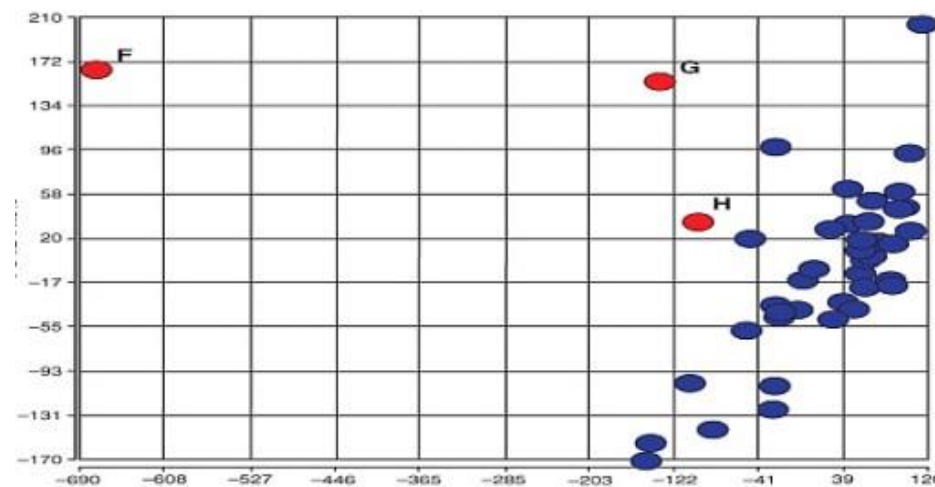
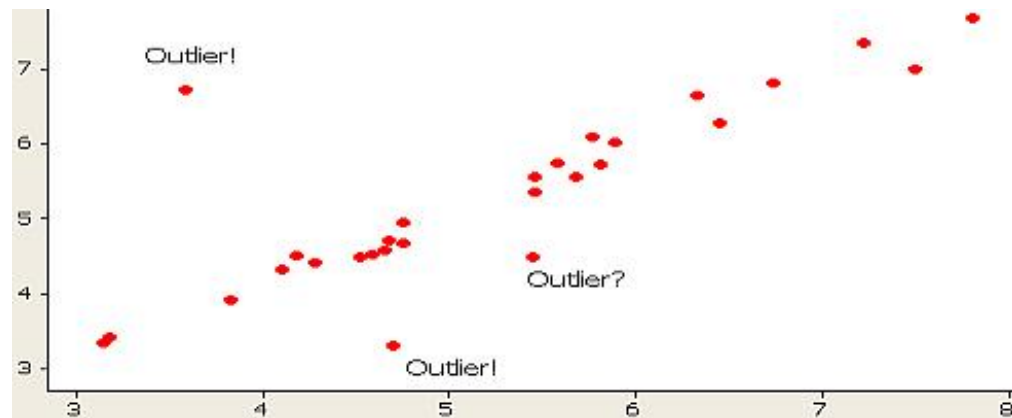
50 → 60 → 80 → 120

- 중요한 정보

- 먼저 원인을 확인해야 함
- 무조건 제거가 아닌 원인 분석 후 판단
- 이상치 탐지는 기술 문제이지만, 제거는 도메인 지식이 필요

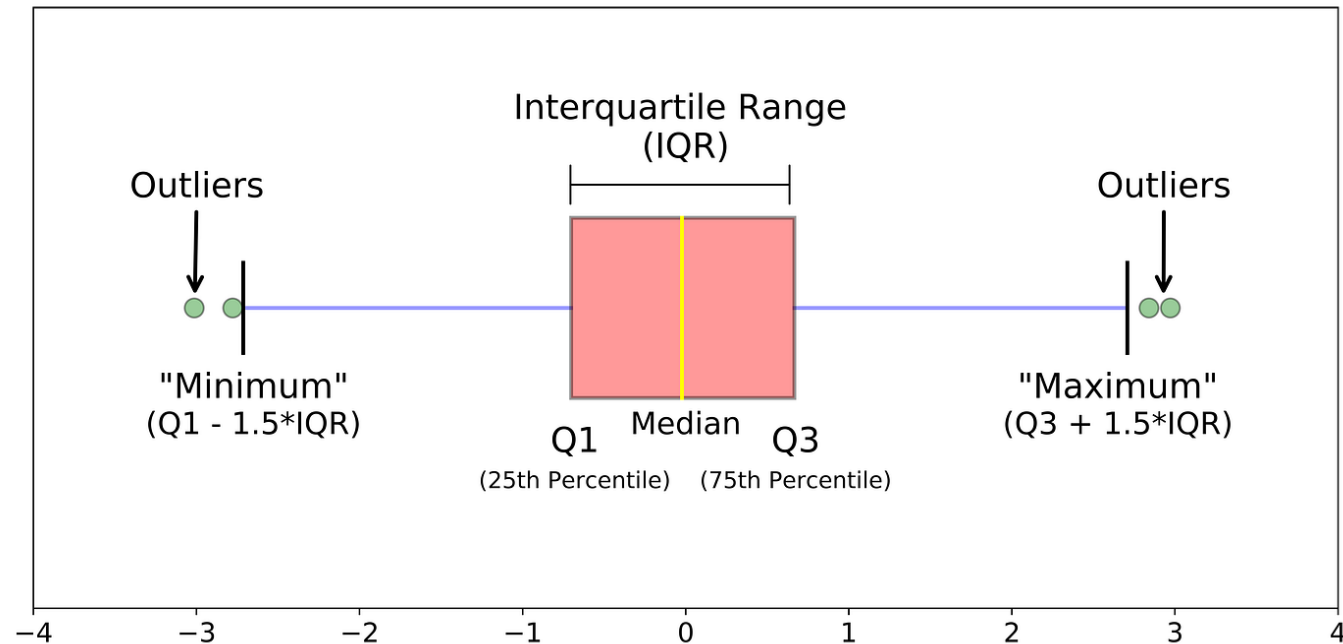
이상치 탐지 방법 1: 시각화

- 산점도



이상치 탐지 방법 2: IQR

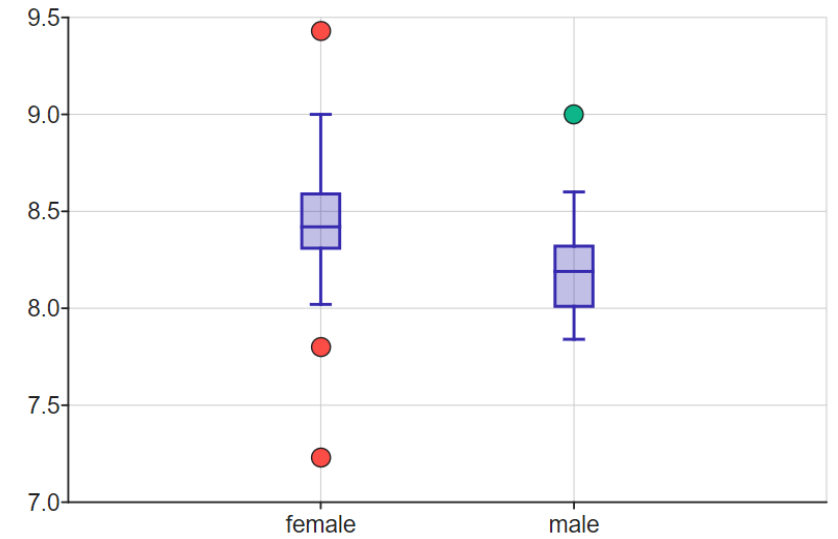
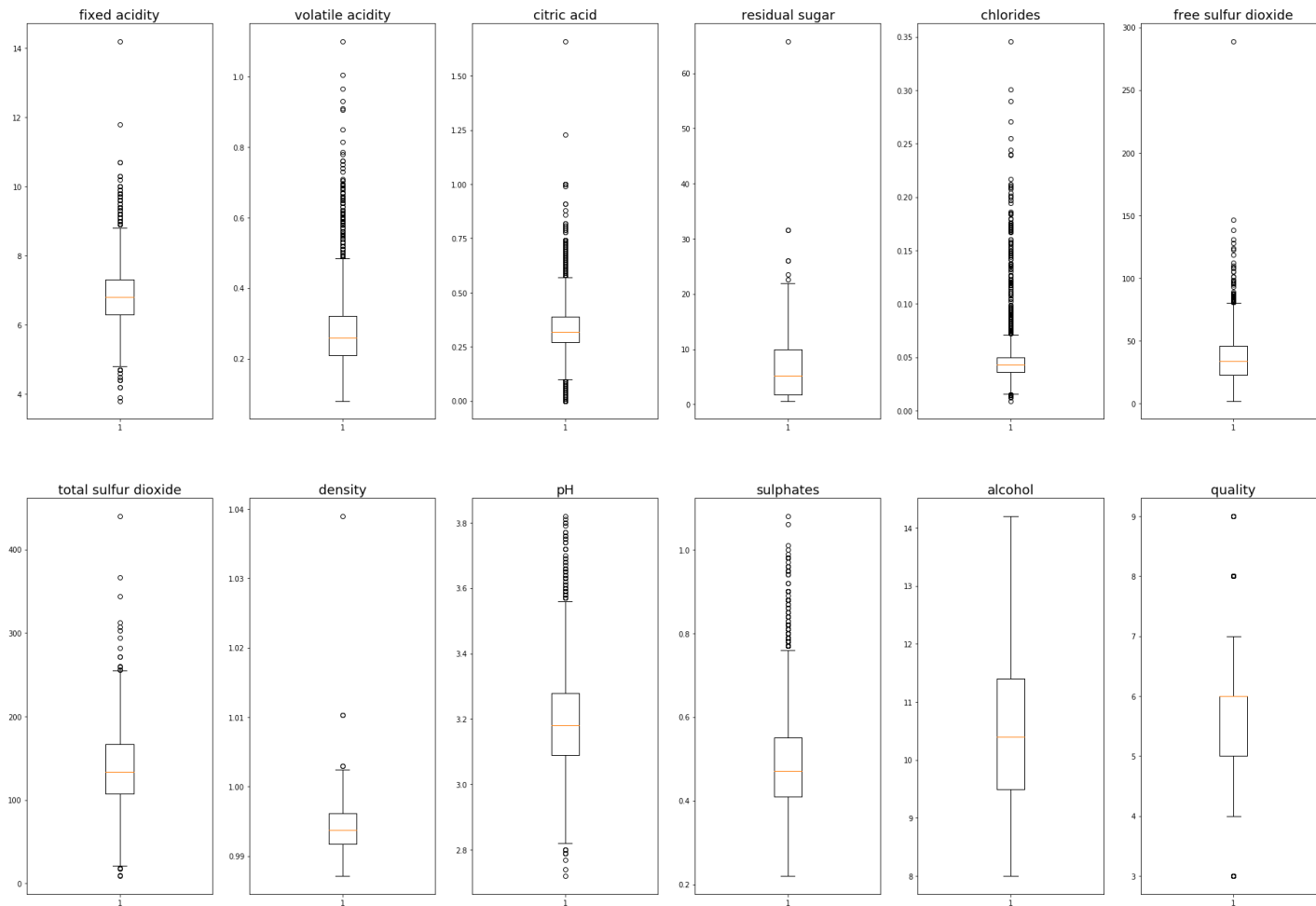
- IQR (Interquartile Range)
- 데이터의 중앙 50% 범위
- 이상치 판단 기준: $Q1 - 1.5 \times IQR$ 보다 작거나 $Q3 + 1.5 \times IQR$ 보다 크면 이상치



- IQR은 가장 널리 사용되는 이상치 탐지 방법이다

이상치 탐지 방법 2: IQR

Boxplots of each feature showing outliers



이상치 제거 방법

- 방법 1: 행 삭제

온도		온도
35		35
36		36
500	→	37
37		38
38		

- 간단하지만 정보 손실이 생긴다

이상치 제거 방법

- 방법 2: 값 대체

온도		온도
35		35
36		36
500	→	36
37		37
38		38

- 평균값, 중앙값, 최근 값 등으로 대체
- 장점: 데이터 수 유지
- 단점: 실제 정보 손실 가능

이상치 처리 절차

- 이상치 탐지
 - 활용 방법: 시각화, Box Plot
- 원인 분석
 - 이 값은 오류인가, 중요한 신호인가?
- 처리 방법 결정
 - 원인에 따라 처리 방법 선택
 - 센서 오류 → 삭제, 평균/중앙값 대체
- 실제 공정 이상
 - 유지
 - 별도 분석 수행
- 결과 확인
 - 처리 전후 비교

What is Learning?

- In machine learning, learning means improving performance on a task through experience/data.
- A computer program is said to learn from experience E with respect to some task T and some performance measure P ,
if its performance on T , as measured by P , improves with experience E . (*Tom Mitchell*)
- Instead of being explicitly programmed with rules, a system learns patterns from data and improves its predictions or decisions over time.

Learning Paradigms

- **Supervised Learning**

Learn from labeled data (x, y)

- **Unsupervised Learning**

Discover patterns or structure from unlabeled data

- **Semi-Supervised Learning**

Learn from a small amount of labeled data and a large amount of unlabeled data

- **Self-Supervised Learning**

Automatically generate supervision signals from the data itself

- **Reinforcement Learning**

Learn optimal actions through interaction with an environment and reward signals

Machine Learning Tasks

- **Regression**
Predict continuous values
- **Classification**
Predict discrete categories
- **Clustering**
Group similar data points without labels
- **Generation**
Generate new data samples
- **Reconstruction, Recommendation, Anomaly Detection, ...**

Supervised Learning

- Supervised learning is a machine learning approach where a model learns from labeled data
- Each training example contains:
 - Input (features)
 - Output (label)
- The goal is to learn a relationship between the input and the output

Supervised Learning

- Learning Process
 1. Provide a dataset with input-output pairs
 2. The model learns patterns from the data
 3. The trained model predicts outputs for new inputs
- Example

Task: email spam detection

Input: email text

Output: spam or not spam

Notation

Common notation in deep learning

- a : A scalar (integer or real)
- $\mathbf{a}$: A vector
- $\mathbf{A}$: A matrix
- $\mathbb{A}$ or $\mathcal{A}$: A set

Mathematical Formulation of Supervised Learning

- Training Data

We are given a dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$

- x_i : input feature
- y_i : target label
- N : number of training samples

- Model

We learn a function

$$f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$$

- f : model
- θ : model parameters

Mathematical Formulation of Supervised Learning

- The goal is to find parameters that minimizes prediction error:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N L(f_{\theta}(x_i), y_i)$$

- $L(\cdot)$: loss function

Loss

- A loss function measures how different the model's prediction is from the true answer
→ quantifies how wrong the model is
- Why Do We Need Loss?
 - To train a model, we need a way to evaluate its predictions
 - The loss function tells the model:
 - how good the prediction is
 - how much error it made
 - how the model should improve
 - The training process aims to minimize the loss

Loss

- Example

True label: Cat

Model prediction: Dog

→ The loss will be large

Ture label: Cat

Model prediction: Cat

→ The loss will be small

- Training Objective

During training, the model adjusts its parameters to reduce the loss over the training data

Common Loss Functions

- Mean Squared Error (MSE)

Used for regression problems

It measures the average squared difference between the predicted value and the true value.

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- y_i : true value
- $\hat{y}_i$: predicted value

Large errors are penalized more strongly because the difference is squared.

Example tasks:

- house price prediction
- Temperature forecasting

One-Hot Encoding

- In classification problems, class labels are often represented as vectors rather than single numbers.
- One-hot encoding converts a class label into a vector representation.
- Example
three classes: cat, bird, and dog
- One-hot encoding represents them as
Cat $\rightarrow [1, 0, 0]$
Bird $\rightarrow [0, 1, 0]$
Dog $\rightarrow [0, 0, 1]$

One-Hot Encoding

- Properties
 - The vector length equals the number number of classes
 - Only one element is 1
 - All other elements are 0
- Why do we use one-hot encoding in classification?
 1. If class labels are represented as scalars (e.g., 0, 1, 2) and the model is trained using MSE, a serious problem arises
 - Artificial ordering is introduced between independent classes
 - The distance between class labels become meaningful, even though classes are categorical

For example, if classes are encoded as:
Cat = 0, Bird =1, Dog = 2

the model implicitly assumes: Cat is closer to Bird than Dog, which is not meaningful for categorical features
 2. One-hot encoding allows the model to represent labels as probability distributions → for cross entropy loss

Common Loss Functions

- Cross-Entropy Loss

Used for classification problems

It measures how different the predicted probability distribution is from the true label

$$L = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

- y_k : true label (one-hot encoding)
- $\hat{y}_k$: predicted probability

The loss becomes large when the model assigns low probability to the correct class

Example tasks:

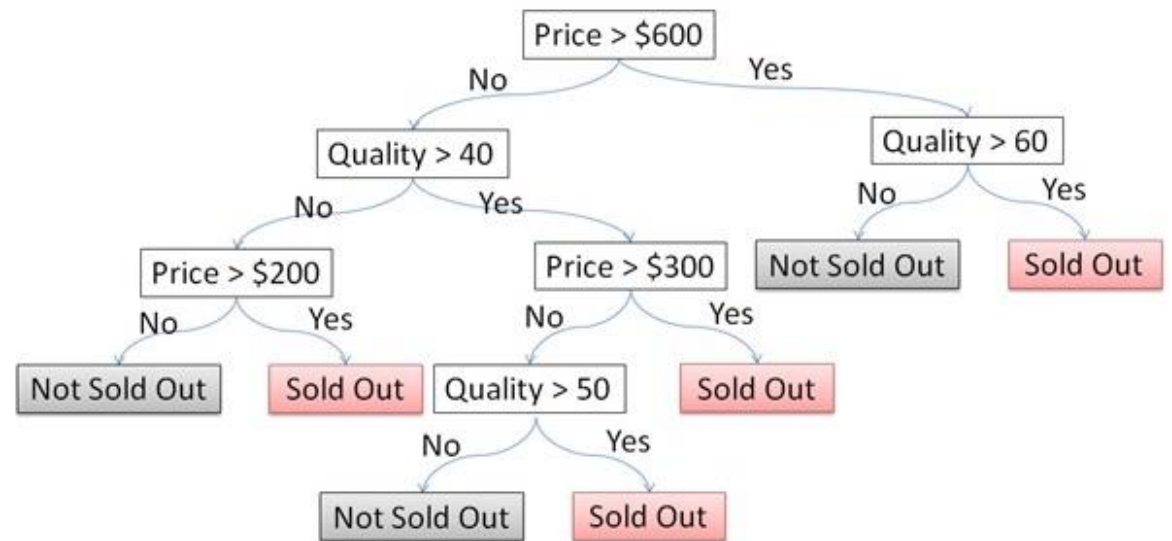
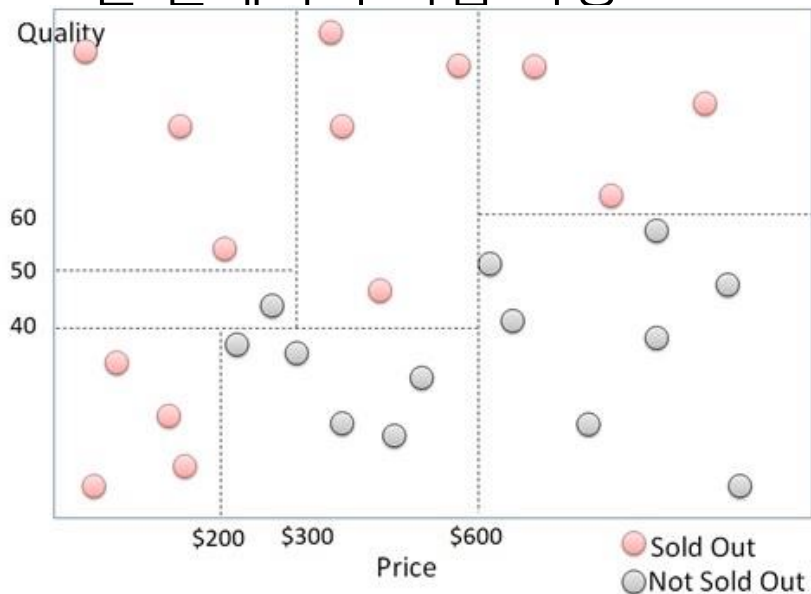
- image classification
- spam detection

XGBoost

- 제조 데이터의 특징
 - 표(Table) 형태 데이터
 - 변수 수가 많음
 - 결측치 존재
 - 비선형 관계 존재
- XGBoost는 표 형식 데이터 분석에 매우 강력하다

트리 기반 모델

- 트리 기반 모델(Tree-based Model)은 데이터를 반복적으로 분할(split)하면서 예측을 수행
- 각 노드(node)에서 모델은 특정 변수(feature)와 임계값(threshold)을 선택하여 데이터를 두 개의 그룹으로 나눔
- 이 과정의 목표는 가능한 한 동일한 특성을 가진 데이터들로 구성된 순수한(pure) 그룹을 만드는 것
- 여러 번의 분할을 조합함으로써, 트리 기반 모델은 데이터에 존재하는 복잡하고 비선형적인 관계까지 학습 가능

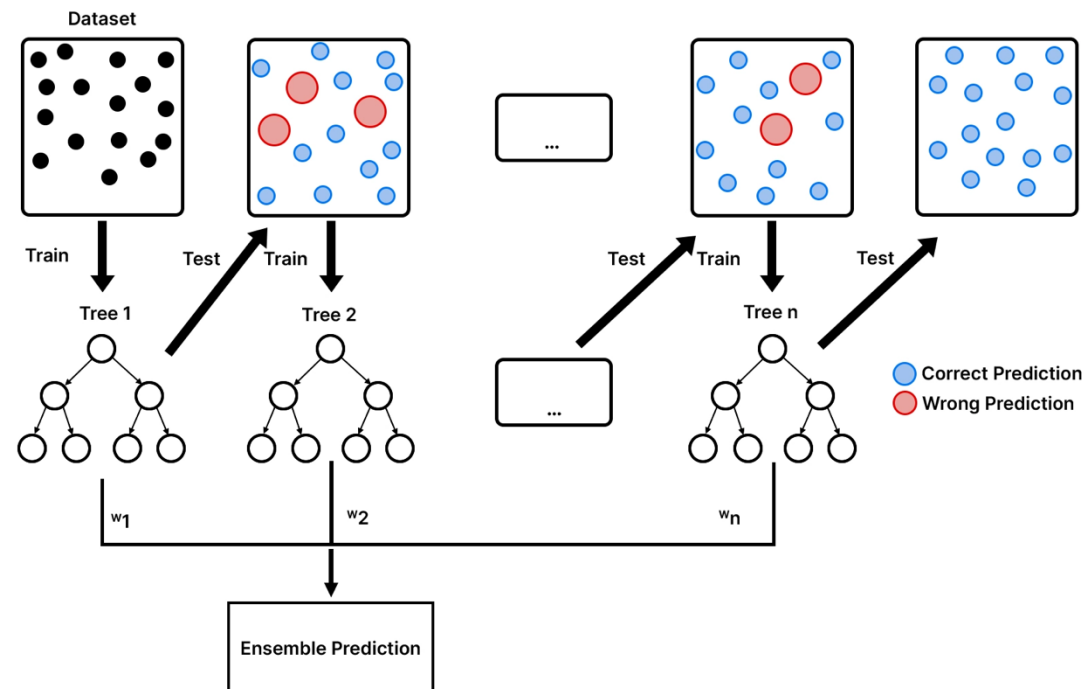


단일 트리기반 모델의 한계

- 하나의 트리만 사용하면
 - 예측 성능 제한
 - 데이터 변화에 민감
- 해결 방법
 - 여러 개의 트리를 함께 사용 → 앙상블(Ensemble)
- 여러 모델을 결합하면 더 정확해진다

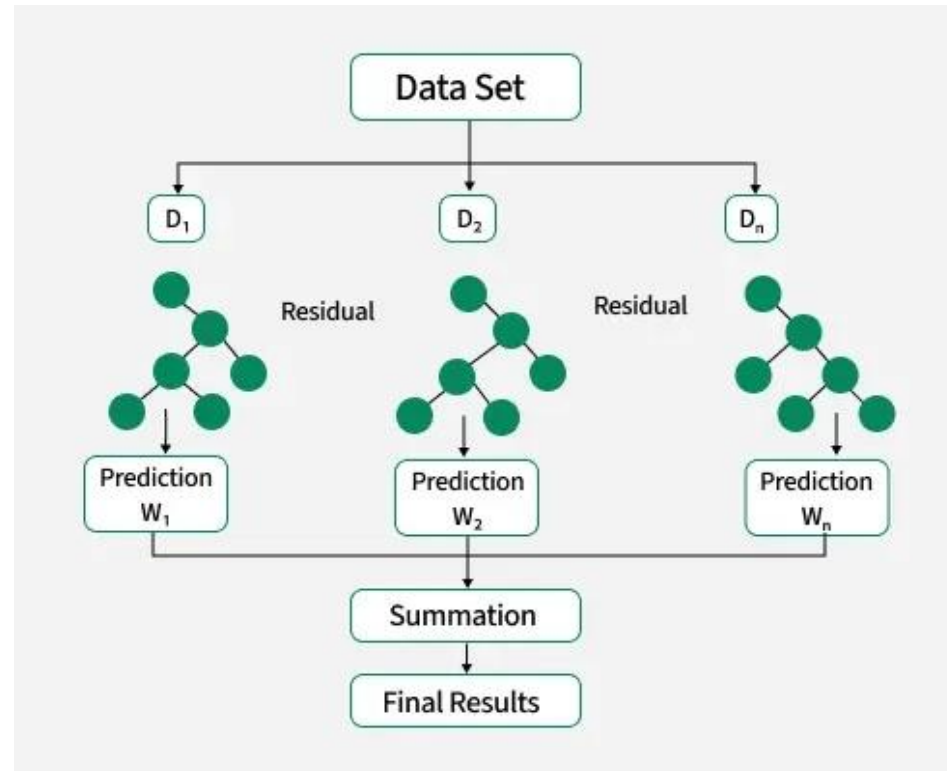
XGBoost

- XGBoost는 트리를 순차적으로 생성하며, 새로운 트리는 이전 트리들이 만들어낸 오차를 보완하도록 학습된다. 즉, 각 트리는 남아 있는 예측 오차를 줄이기 위해 추가된다.
- 이와 같이 여러 개의 약한 학습기(Weak Learner)를 결합함으로써, XGBoost는 높은 예측 성능을 갖는 강력한 모델을 구축할 수 있다.



XGBoost

- XGBoost는 여러 개의 트리가 생성한 예측 결과를 가중합(weighted sum)하여 최종 예측을 수행한다.
- 즉, XGBoost는 성능이 약한 여러 개의 트리(Weak Trees)를 결합하여 높은 예측 성능을 갖는 강력한 모델(Strong Model)로 만들어 준다.



변수 중요도(Feature Importance)

- **XGBoost의 장점**

- 어떤 변수가 중요한지 확인 가능

변수	중요도
온도	42%
압력	28%
진동	20%
전류	10%

- 해석

- 온도가 품질에 가장 큰 영향
- 제조 현장에서 매우 유용

MLP vs XGBoost

항목	MLP	XGBoost
데이터 양	많이 필요	상대적으로 적음
해석 가능성	낮음	높음
학습 속도	보통	빠름
표 형식 데이터	보통	매우 우수

- 많은 경우 XGBoost가 효과적이나 데이터셋 또는 상황에 따라 다름
→ 첫 시도로 XGBoost를 추천